

UNIVERSIDAD POLITÉCNICA DE MADRID

**ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN**



GRADO EN INGENIERÍA BIOMÉDICA

TRABAJO FIN DE GRADO

**DESIGN AND DEVELOPMENT OF AN ENVIRONMENT
FOR THE VISUALIZATION OF BIOMECHANICAL
INFORMATION OF PHYSIOTHERAPY PATIENTS**

**BELÉN GÓMEZ MARTÍNEZ
JUNIO 2026**

TRABAJO DE FIN DE GRADO

Título: Diseño y desarrollo de un entorno para la visualización de información biomecánica de pacientes de fisioterapia

Título (inglés): Design and development of an environment for the visualization of biomechanical information of physiotherapy patients

Autor: Belén Gómez Martínez

Tutor: Rodrigo García Carmona

Departamento: Departamento de Ingeniería de Sistemas Telemáticos

MIEMBROS DEL TRIBUNAL CALIFICADOR

Presidente: —

Vocal: —

Secretario: —

Suplente: —

FECHA DE LECTURA:

CALIFICACIÓN:

UNIVERSIDAD POLITÉCNICA DE MADRID

**ESCUELA TÉCNICA SUPERIOR DE
INGENIEROS DE TELECOMUNICACIÓN**
Departamento de Ingeniería de Sistemas Telemáticos



TRABAJO FIN DE GRADO

**DESIGN AND DEVELOPMENT OF AN
ENVIRONMENT FOR THE VISUALIZATION OF
BIOMECHANICAL INFORMATION OF
PHYSIOTHERAPY PATIENTS**

Belén Gómez Martínez

Junio 2026

Resumen

Este Trabajo de Fin de Grado presenta el desarrollo de CHLOE (Clinical Helper for Locomotion Objective Evaluation), una aplicación web para la visualización y el análisis de datos biomecánicos que forma parte del proyecto LibreMotion, cuyo objetivo es democratizar la captura, el almacenamiento y el análisis de datos biomecánicos, especialmente los obtenidos con dispositivos de bajo coste.

CHLOE pretende mejorar el acceso y uso por parte de clínicos de datos de movimiento registrados en formato C3D, el estándar en laboratorios de biomecánica para almacenar trayectorias de marcadores y señales analógicas complementarias, como puede ser la electromiografía (EMG). Este formato suele requerir software especializado, complejo de configurar y, en ocasiones, poco intuitivo. Además, la mayor parte de las funcionalidades que proporcionan los visores de C3D no obedecen a las necesidades de los fisioterapeutas. Por ello, se ha implementado una solución basada en tecnologías de código abierto para su ejecución en navegadores web, fácil de usar, que no requiera prácticamente ninguna configuración y se centre en las tareas que un fisioterapeuta suele desarrollar con más frecuencia en la práctica clínica. Como su uso en el día a día exige muy poco tiempo por parte del profesional médico y resulta fácil de aprender, su integración en procesos de trabajo ya existentes será relativamente fácil.

CHLOE reconoce automáticamente los datos en formato C3D correspondientes al protocolo Plug-in Gait, el estándar más común para la captura de movimiento (MoCap) del cuerpo humano completo, pero no se limita a este modelo, permitiendo visualizar cualquier conjunto de marcadores. Esto permite integrar varios orígenes de datos, que van desde sistemas de captura de movimientos ópticos (OMC), típicos de entornos de investigación, hasta dispositivos de bajo coste como sensores inerciales o dispositivos de realidad extendida (XR).

Este Trabajo Fin de Grado utiliza Python y la biblioteca ezc3d para transformar los datos binarios, propios de C3D, en estructuras JSON. Estos JSON son, a su vez, consumidos por un motor de visualización desarrollado con Three.js, que permite renderizar nubes de puntos animadas, construir esqueletos dinámicos si los datos de entrada se corresponden con los del protocolo Plug-in Gait y realizar un mapeo de avatares tridimensionales provenientes de Mixamo. Para representar en forma de gráfica los datos que seleccione el usuario, se utiliza

Plotly.js. Esta interfaz gráfica también permite seleccionar marcadores concretos para ver su trayectoria. Además, la herramienta integra la gestión de datos analógicos para evaluar parámetros como fuerzas de reacción y activación muscular.

El despliegue de la aplicación se ha realizado mediante contenedores Docker, garantizando la portabilidad del sistema y una instalación sencilla. Finalmente, el sistema ha sido validado mediante pruebas de usabilidad con profesionales de la fisioterapia, quienes han confirmado su utilidad en entornos clínicos y educativos, destacando su potencial para la imáginería motora y el análisis biomecánico objetivo.

Palabras clave: Ingeniería biomédica, biomecánica, captura de movimientos, C3D, Plug-in Gait, visualización de datos, datos analógicos, fisioterapia.

Abstract

This Bachelor Thesis presents the development of CHLOE (Clinical Helper for Locomotion Objective Evaluation), a web-based application designed for the visualization and analysis of biomechanical data. CHLOE is integrated into the LibreMotion project, which aims to democratize the capture, storage, and analysis of biomechanical data, with a particular focus on data obtained from low-cost devices.

The primary goal of CHLOE is to improve clinical access to movement data recorded in C3D format, the standard in biomechanics laboratories for storing marker trajectories and complementary analog signals, such as electromyography (EMG). Standard C3D viewers often require specialized software that is complex to configure and non-intuitive; furthermore, their functionalities rarely align with the specific needs of physiotherapists. To address this, a solution based on open-source technologies was implemented. CHLOE is a browser-based, easy-to-use tool that requires virtually no configuration and focuses on the tasks most frequently performed by physiotherapists in clinical practice. Its ease of use and low learning curve facilitate seamless integration into existing clinical workflows.

CHLOE automatically recognizes C3D data corresponding to the Plug-in Gait protocol, the most common standard for full-body motion capture (MoCap). However, it is not limited to this model and can visualize any set of markers. This flexibility allows for the integration of diverse data sources, ranging from optical motion capture (OMC) systems used in research to low-cost devices such as inertial sensors or extended reality (XR) headsets.

Technically, the project utilizes Python and the ezc3d library to transform binary C3D data into JSON structures. These files are consumed by a visualization engine developed with Three.js, which enables the rendering of animated point clouds, the construction of dynamic skeletons (for Plug-in Gait data), and the mapping of 3D avatars from Mixamo. Plotly.js is used to generate interactive graphs of user-selected data, while the graphical interface also allows for tracking specific marker trajectories. Additionally, the tool integrates analog data management to evaluate parameters such as ground reaction forces and muscle activation.

The application was deployed using Docker containers to ensure system portability and straightforward installation. Finally, the system was validated through usability testing with physiotherapy professionals, who confirmed its utility in clinical and educational settings,

highlighting its potential for motor imagery and objective biomechanical analysis.

Keywords: Biomedical engineering, biomechanics, motion capture, C3D, Plug-in Gait, data visualization, analog data, physiotherapy.

Agradecimientos

Y con esto, los cuatro años más cambiantes de mi vida llegan a su fin.

Una niña, sin carnet, ni siquiera con la mayoría de edad, que siempre había ido al mismo colegio. Esa era yo cuando entré por la puerta de la ETSIT por primera vez, rodeada de desconocidos y abrumada por tanta información.

En un momento han pasado cuatro años. No ha pasado nada y, a la vez, ha pasado de todo. He crecido en todos los sentidos. Siempre dicen que la etapa universitaria te cambia, pero jamás hubiese imaginado cuánto. Desde la rutina de ir al parking del A, esperar a que esa barrera se levantara o intentar no rayar ningún coche (porque vaya parking más estrecho), esperar en las escaleras a mis Nenukis, subir a clase... o no, o esos días de prácticas de laboratorio en los que terminaban echándonos de la universidad porque tenían que cerrar. En su momento te quejas de la carga o del estrés (soy una quejica, qué le hago), pero reconozco que lo echaré de menos.

Repitiendo la mención, gracias a mis Nenukis, porque no sé qué hubiese sido de estos cuatro años sin vosotras. Cogimos confianza tan rápido por algo. No sabía que se podía echar tanto de menos a alguien hasta que la mitad os fuisteis de Erasmus y la otra mitad nos quedamos aquí, casi sin vernos por la diferencia de itinerarios. Y a las que habéis compartido conmigo los últimos días en la ETSIT, ¿cómo voy a trabajar ahora con otras personas, si casi podríamos compartir CV?

Gracias a mis padres por el apoyo que me han dado, me dan y me darán siempre. Sé que estábais deseando tanto como yo este momento, y sé que siempre habéis sabido que lo conseguiría. Gracias por recordármelo cuando más dudaba de mí misma.

Gracias, Rodrigo, por esta oportunidad. Ha sido un placer trabajar contigo este año.

Por último, gracias a mí misma, porque aunque dudé muchas veces, no me rendí.

Ahora sí, esto termina aquí, deseando ver qué viene después.

Contents

Resumen	I
Abstract	III
Agradecimientos	V
Contents	VII
List of Figures	XI
List of Tables	XIII
1 Introduction	1
1.1 Context	1
1.2 State of Art	2
2 Enabling Technologies	5
2.1 C3D File Format and Biomechanical Data Standards	5
2.2 Backend Technologies	8
2.2.1 Biomedical Data Parsing: ezc3d	8
2.2.2 Python Ecosystem	8
2.3 Frontend Technologies	9
2.3.1 Three.js Framework and FBX	9
2.3.2 Mixamo Character Assets	10
2.4 Data Visualization	11
2.4.1 Plotly.js	11
2.5 Deployment Infrastructure	12
2.5.1 Docker Containerization	12
3 Requirements	13
3.1 Functional Requirements (FR)	13
3.2 Non-Functional Requirements (NFR)	14

4	Design and Implementation	17
4.1	System Architecture	17
4.1.1	Directory and File Organization	18
4.1.2	Environment Configuration and Deployment	19
4.2	Data Ingestion and Processing Pipeline	19
4.2.1	Asynchronous File Ingestion and Synchronization	19
4.2.2	Exception Handling and Data Integrity	20
4.2.3	Biomechanical Data Standardization	21
4.2.4	Analog Signal Classification	21
4.2.5	Data Orchestration and Output Structure	22
4.3	Real-time Visualization Engine	23
4.3.1	System Initialization	23
4.3.2	The Animation Loop and Temporal Synchronization	24
4.3.3	Conditional Updates	25
4.3.4	Continuous State Synchronization	25
4.4	Kinematic Mapping and Avatar Animation	26
4.4.1	Standardization via Plug-in-Gait Protocol	26
4.4.2	Skeleton Mode	29
4.4.3	Avatar Mode	30
4.4.3.1	Bone Alignment Algorithm (alignBone)	30
4.4.3.2	Root Placement and Global Orientation	31
4.4.3.3	Kinematic Chain: Marker Pairs per Bone	31
4.5	Analytical and Visualization Features	34
4.5.1	Dynamic Marker Trajectories	34
4.5.2	Interactive 3D Vectors and Angular Analysis	35
4.5.3	Analog Signal Visualization	36
5	Validation	37
5.1	Validation Methodology	37
5.2	C3D Format Compatibility Testing (Test Suites)	37
5.2.1	Sample01 - ADTech/DEC and Sample02 - Vicon 370	38
5.2.2	Sample08 - Parameter and Data Block Location Variants	38
5.2.3	Sample36 - POINT:FRAMES Integer vs. Floating-Point	39
5.2.4	Compatibility Test Results	39
5.3	Current Manufacturers Files (Sample00)	39
5.3.1	Vicon Motion Systems	39
5.3.2	NextGen Ergonomics	40

5.3.3	Motion Analysis Corporation (MAC)	40
5.3.4	Innovative Sports Training (IST) and Codamotion	40
5.3.5	Advanced Realtime Tracking GmbH (ART)	41
5.3.6	Cometa Systems	41
5.3.7	Manufacturer Test Results	41
5.4	Robustness and Formatting Issues	41
5.4.1	Sample12 - Stress Test: 364 Markers, 18,178 Frames	41
5.4.2	Sample13 - Files with Parameter Errors	42
5.4.3	Sample16 and Sample20 (Invalid Data)	42
5.4.4	Sample18 - Corrupt Parameter Section	42
5.4.5	Sample21 - Data/Parameter Rate Mismatch	43
5.4.6	Robustness Test Results	43
5.5	Clinical and Functional Validation with Physiotherapy Professionals	43
5.5.1	Usability Tests	43
5.5.2	Qualitative Findings	44
5.5.3	Tested Devices	45
5.6	Summary	45
6	Conclusions and Future Work	47
6.1	Conclusions	47
6.2	Future Work	49
	Appendix A Impact of the Project	i
A.1	Social Impact	i
A.2	Ethical and Privacy Considerations	ii
A.3	Economic Impact	ii
A.4	Environmental Impact	ii
A.5	Professional Responsibility	iii
	Appendix B Economic Budget	v
B.1	Technological Resources	v
B.2	Human Resources	v
B.3	Total Cost	vi
	Bibliography	vii

List of Figures

2.1	Plug-in-Gait protocol markers.	7
2.2	Mixamo avatar with and without the mesh, so the rig can be seen.	11
4.1	Sequence diagram of the C3D ingestion pipeline and error handling logic. . .	20
4.2	CHLOE main interface initialization with a 3D viewport and control sidebars.	23
4.3	Sequence diagram of the real-time animation loop and telemetry synchronization.	24
4.4	Comparison between skeleton mode (a) and avatar mode (b) for the same frame of motion capture data.	26
4.5	3D visualization of motion capture markers and their trajectories over time. .	34
4.6	Vector-based analysis between the left forearm and left wrist markers, including the decomposition of the vector into its Cartesian projections for quantitative motion assessment.	35
4.7	Synchronized visualization of 3D motion and EMG signals using a shared temporal reference.	36

List of Tables

3.1	Summary of Functional and Non-Functional Requirements.	15
4.1	Standard marker nomenclature equivalences across motion capture manufacturers. The Plug-in Gait (Vicon Nexus) label is the reference standard used throughout CHLOE. Markers marked with * do not appear in the official Plug-in Gait documentation.	27
4.2	Hand and finger marker set used in CHLOE, following the LibreMotion naming convention. Each marker label encodes laterality (R/L), finger identity, and phalange level.	28
4.3	Kinematic chain: Origin marker → Target marker → Mixamo bone driven. .	32
4.4	FINGER_MARKERS mapping: each phalange pair and its corresponding Mixamo bone.	33
5.1	Individual SUS questionnaire results.	44
5.2	Traceability matrix: Requirements vs. Validation outcomes.	46
B.1	Estimated total cost of the project.	vi

Introduction

1.1 Context

Clinical biomechanics is the discipline that applies mechanical principles to the study of human movement for diagnostic, therapeutic, and preventive purposes. Over recent decades, motion analysis has evolved from subjective clinical observation to objective assessment based on high-precision Motion Capture (MoCap) systems. These systems record three-dimensional marker trajectories and complementary physiological signals, like electromyography (EMG) or ground reaction forces (GRF), typically in the C3D format, which has become the *de facto* industry standard for storing and exchanging biomechanical data.

The objective analysis of movement offers enormous clinical value: it enables the identification of pathological gait patterns, the monitoring of rehabilitation progress, and the support of evidence-based clinical decisions. However, there is a significant gap between the data generated in biomechanical laboratories and its actual use in daily clinical practice. Transforming raw C3D data into clinically interpretable information currently requires a complex, multi-stage pipeline (encompassing model scaling, inverse kinematics, inverse dynamics, muscle force estimation, and forward simulation) that demands both specialized software and deep technical expertise. For a physiotherapist in a clinical setting, this overhead is often prohibitive: the clinician does not need to compute joint torques or simulate muscle activations; they simply need to see the movement and correlate it with the

accompanying physiological signals immediately.

This disconnect, commonly referred to as the “clinical gap”, motivates the need for a lightweight, accessible visualization tool that bypasses the complexity of full biomechanical modeling and adopts a drag-and-drop philosophy. By removing the burden of complex biomechanical modeling, it would allow clinicians to focus on visual interpretation and direct data analysis, facilitating its use on any device via a web browser.

This work addresses that need through the design and development of CHLOE (Clinical Helper for Locomotion Objective Evaluation), a browser-based application for the visualization and analysis of biomechanical data recorded in C3D format, within the framework of the LibreMotion project, which aims to democratize the capture, storage, and analysis of movement data with particular attention to low-cost devices.

1.2 State of Art

Several software solutions currently exist for working with biomechanical C3D data, yet none fully address the clinical accessibility requirements outlined above.

OpenSim [1] is the reference standard in biomechanical research. It is a powerful open-source platform that supports the construction of detailed musculoskeletal models, dynamic simulation, and the estimation of muscle forces. Its processing pipeline (comprising scaling, inverse kinematics, inverse dynamics, static optimization, and forward dynamics) is essential for high-level research but constitutes a significant barrier to routine clinical use. Notably, OpenSim does not natively support C3D files as direct input [2]. Raw motion capture data must first be converted into OpenSim-compatible formats (TRC (Track Row Column) for marker trajectories [3] and MOT (Motion) for analog/force data [4]). This conversion step typically requires intermediate tools such as MATLAB or Python-based APIs (Application Programming Interface), such as BTK (Biomechanical ToolKit, see below) or ezc3d (see Section 2.2.2), which increases the technical burden on clinical users. Beyond format compatibility, it requires a dedicated workstation, considerable expertise in biomechanical modeling, and substantial time investment per patient trial.

Visual3D (C-Motion) [5] is a widely used commercial alternative in clinical gait laboratories. It offers advanced analysis capabilities, including the computation of joint kinematics and kinetics, and supports C3D import natively. However, its licensing cost, steep learning curve, and desktop-only nature limit its adoption outside well-resourced research centers.

Vicon Nexus [6] is the flagship software from the leading MoCap manufacturer. While tightly integrated with Vicon equipment and the Plug-in Gait protocol, these tools are not hardware-agnostic: they are designed primarily as data collection and post-processing environments rather than as flexible clinical viewers. Their use is restricted to institutions

that have invested in the corresponding proprietary infrastructure.

BTK [7] is the historically dominant open-source framework for C3D access and has served as the reference tool for the biomechanics community for many years. It provides programmatic access to C3D data, but it offers no graphical interface for clinical users.

A common limitation across all existing solutions is the absence of synchronized visualization between 3D motion and physiological analog signals within a single, accessible, browser-based environment. Furthermore, most tools either target research workflows, which require configuration and modeling knowledge, or are tied to specific hardware vendors.

This gap in the landscape of available tools justifies the development of a new system: one that is hardware-agnostic, browser-deployable without local installation, focused on the tasks most relevant to physiotherapy practice, and capable of rendering both three-dimensional movement and analog signals in a synchronized, interactive interface with minimal user friction.

Enabling Technologies

This chapter presents the core technologies that enable CHLOE’s functionality, from biomechanical data processing to 3D visualization and deployment infrastructure. For each component, the rationale behind its selection is discussed alongside the available alternatives in order to justify the architectural decisions made during the design phase.

2.1 C3D File Format and Biomechanical Data Standards

The choice of which data format to support is the most relevant design decision for CHLOE. The biomechanics community has historically converged around two main formats for storing motion capture data: C3D and TRC/MOT (the pair used by OpenSim, as introduced in Section 1.2). Although this separation is transparent for research pipelines that feed directly into OpenSim, it introduces friction in clinical settings where data from different sources must be handled as a single, self-contained recording.

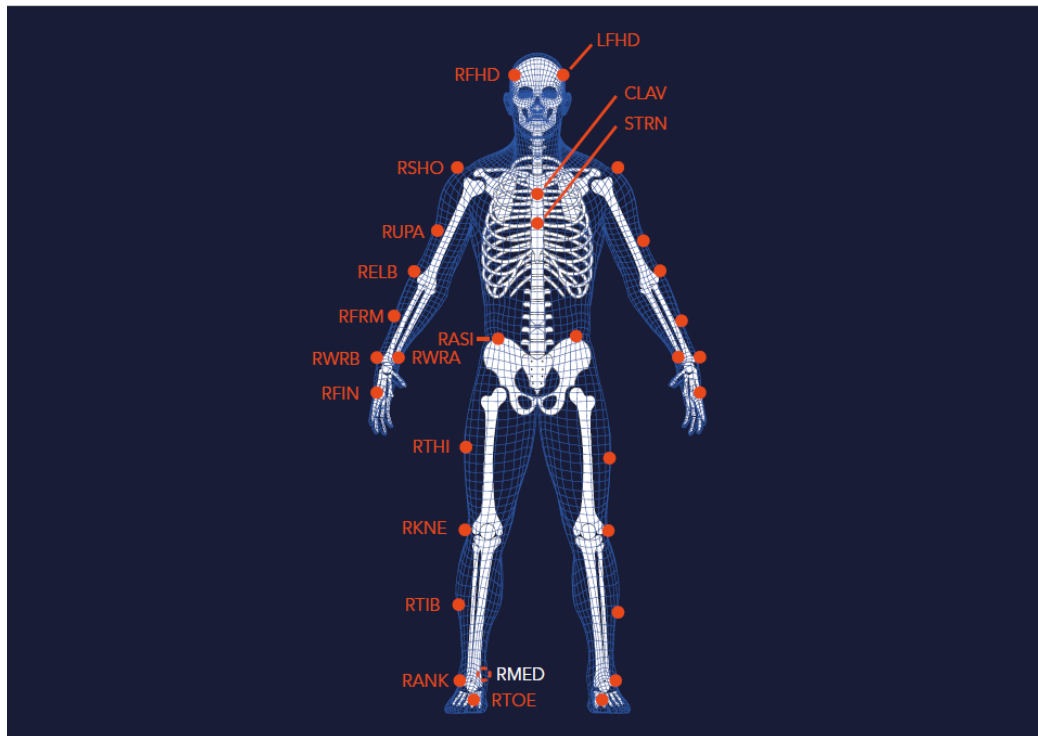
The C3D format [8] serves as the basis for storing and exchanging biomechanical motion data in CHLOE. Originally developed in the 1980s, C3D, stores synchronized 3D marker trajectories and analog signals (EMG, ground reaction forces, and other physiological channels) within a single binary file. C3D has become the *de facto* standard in biomechanical laboratories, sports medicine, rehabilitation facilities, and motion capture studios across the world.

However, despite its widespread adoption, C3D presents significant implementation variability that poses critical challenges for software interoperability:

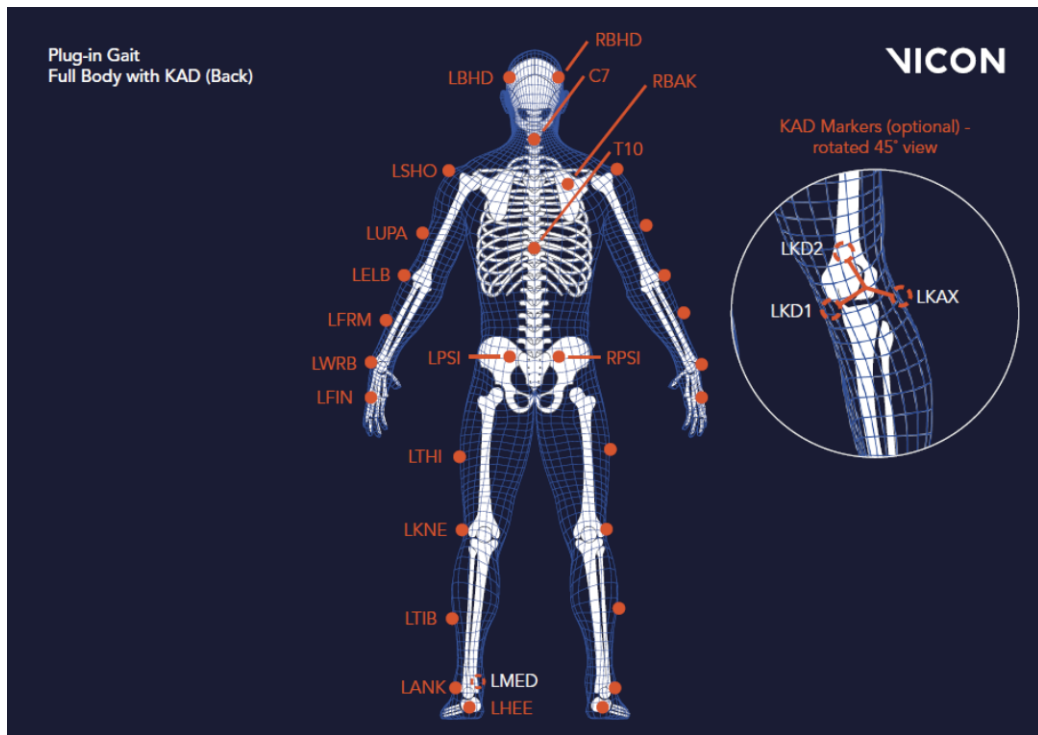
- **Nomenclature Heterogeneity:** Marker naming conventions are not universally standardized. Instead, they depend on both the capture software (Vicon, OptiTrack, etc.) and the biomechanical protocol employed (Plug-in Gait, Helen Hayes, etc.). A single anatomical point may appear with different subject prefixes or abbreviations across systems, requiring prior normalization for automatic mapping.
- **Variable Data Structures:** While the system expects a defined column structure for coordinates (X, Y, Z), some files include additional residual columns or calculated data (virtual markers, angles, power metrics) varying the number of columns depending on the source software.
- **File Architecture and Typing:** C3D stores critical parameters, such as frame counts, in both integer and floating-point formats, generating read errors in libraries that do not accommodate both possibilities. Additionally, different endianness variants (Intel, DEC, SGI/MIPS) complicate compatibility with modern processors.
- **Signal Quality:** The presence of null or corrupted values, NaN (Not a Number), in specific markers throughout the trials requires error management to prevent 3D animation collapse, transforming these gaps into arrays manageable by the graphics engines.

Standards from fields other than the biomechanics community were considered. OpenXR [9], in particular, was especially suitable since the kind of motion data it records matches the intended use of the tool. However, despite all the previously stated limitations of C3D, it was decided to use it as the basis for CHLOE nonetheless. The reason is that most medical professionals consulted had large libraries of already recorded data in C3D, and most public repositories also used this format. Therefore, opting for another standard would have severely limited the usefulness of the tool. With that said, C3D limitations were kept in mind during the development of CHLOE to avoid or mitigate the aforementioned implementation pitfalls.

For development and testing, C3D files from the CMU Graphics Lab Motion Capture Database [10] and sample files from the official C3D resource repository [11] were used. The latter in particular provide compatibility testing data across different manufacturer implementations. The project follows the Plug-in Gait protocol established in the Nexus 2.17 documentation, found in the Vicon web [12], as shown in Figure 2.1, for mapping between markers and the human body (represented by the Mixamo rig, see below). To extend standardization across files captured with other software, such as Motion Analysis Corporation, marker correspondences with Vicon nomenclature were established.



(a) Front side view.



(b) Back side view.

Figure 2.1: Plug-in-Gait protocol markers. Adapted from the Plug-in Gait Reference Guide, by Vicon Motion Systems Ltd [12].

2.2 Backend Technologies

2.2.1 Biomedical Data Parsing: `ezc3d`

The selection of a C3D parsing library is an important architectural decision, as it determines the breadth of file variants supported, the reliability of the extraction pipeline, and the long-term maintainability of the codebase. Three candidate libraries were evaluated: BTK [7], the Python-native `c3d` package [13], and `ezc3d` [14].

As noted in Section 1.2, BTK provides programmatic access to C3D data. However, its development has been effectively discontinued, with no active maintenance since 2016. As a consequence, it lacks support for several modern C3D variants and manufacturer-specific parameter configurations. Relying on an unmaintained library in a research-adjacent project would introduce an unacceptable risk of silent parsing failures as the C3D ecosystem continues to evolve.

The Python-native `c3d` package offers a lightweight, pure-Python implementation that is straightforward to install. However, its performance on large files is constrained by interpreter overhead, and its handling of edge cases in the binary specification is less robust than that of a natively compiled alternative.

`ezc3d` is a C++ library with Python bindings, actively maintained by the Pyomeca organization. It was selected over BTK due to its active maintenance cycle, and over the Python-native `c3d` package due to the performance advantage of its compiled C++ core for large, high-frequency datasets. Additionally, its Python API provides direct, labeled access to both the POINTS and ANALOG sections of the file, integrating naturally with the rest of the Python backend described in Section 2.2.2.

2.2.2 Python Ecosystem

Python [15] was selected as the backend language for two reasons. First, it offers the most mature ecosystem for scientific data analysis, with battle-tested libraries for numerical computation and binary format parsing. Second, compared to alternatives such as Java or C#, which are common in medical software, Python enables significantly faster prototyping and iteration, which is important for a project where requirements evolve with clinical feedback.

The specific libraries that complete the Python backend infrastructure and orchestrate the data pipeline are:

- **NumPy [16]:** Biomechanical data are typically presented as multidimensional arrays (e.g. 4 coordinates $\times$ n markers $\times$ n frames). NumPy facilitates the high-performance manipulation of these large point clouds through optimized computational operations.

No alternative was considered for this role, since NumPy is the universal standard for numerical array processing in Python.

- **Flask [17]:** This technology was chosen because, unlike alternatives like NGINX [18], Flask can execute Python logic in response to user requests. It can serve as the engine that activates the conversion of the C3D files to JSON format for the frontend. FastAPI [19] and Django [20] were considered but ultimately rejected: FastAPI, while performant, introduces asynchronous programming patterns that add complexity without clear benefit, given that C3D processing is CPU (Central Processing Unit)-bound rather than I/O (Input/Output)-bound; Django, on the other hand, brings a large ORM (Object-Relational Mapping)-based framework that is disproportionate for an application with this scope. Flask’s minimal footprint is ideal for the size and complexity of CHLOE.
- **Gunicorn [21]:** For production deployment, Gunicorn provides a production-grade WSGI (Web Server Gateway Interface) server. This is essential, as Flask’s integrated development server is not designed for concurrent requests or real-world environments. Gunicorn was chosen over uWSGI [22] due to its simpler configuration and native compatibility with the Conda/Mamba environment used for deployment (see Section 2.5). It acts as a robust process monitor, managing server resources and ensuring that the biomechanical processing logic remains scalable and resilient.

2.3 Frontend Technologies

2.3.1 Three.js Framework and FBX

For the 3D rendering layer, the two most mature browser-based alternatives are Three.js [23] and Babylon.js [24], with A-Frame [25] occupying a niche focused on WebXR experiences. Babylon.js is a fully-featured game engine with a built-in physics engine, PBR (Physical-Based Rendering) materials pipeline, and integrated GUI (Graphical User Interface) system; capabilities that, while powerful, introduce unnecessary complexity for a clinical viewer whose primary requirement is the accurate real-time animation of a skeletal hierarchy. A-Frame abstracts the scene graph into a declarative syntax similar to HTML (HyperText Markup Language), which simplifies basic scenes but limits the fine-grained programmatic control over bone quaternions and buffer geometries that CHLOE requires.

Three.js, an open-source JavaScript library to create and display animated 3D graphics in web browsers, occupies the ideal middle ground: It provides direct access to WebGL (Web Graphics Library) primitives, an active maintenance ecosystem, and well-documented

support for FBX model (see below) loading and skeletal animation, but without imposing the overhead typical of a full game engine. In fact, A-Frame is built on top of Three.js.

Node.js [26] was selected for the installation of Three.js and Vite [27]. Vite is a fast development server that provides nearly instantaneous browser reflection of code changes. It also serves as a build tool, packaging and optimizing code into a production-ready version.

Finally, adding models to the scene requires specific loaders. While the Three.js manual recommends GLB/GLTF formats for 3D models due to their efficiency and fast loading characteristics, CHLOE's specific rigging requirements and the complexity of associating marker vectors to skeleton bones made FBX the more functional choice. Specifically, importing avatars in GLB format prevents independent visualization of the skeleton separate from the mesh. FBX proved more suitable for working directly with bones, which is fundamental for establishing constraints and mapping C3D markers.

2.3.2 Mixamo Character Assets

For the humanoid avatar, two main approaches were considered: procedurally generating a skeleton from the marker data directly or mapping the data onto a pre-existing rigged character. The first approach offers complete control but requires significant investment in mesh skinning and rig design; an expertise outside the field of biomedical engineering, out of the scope of this project, and unnecessary given the availability of high-quality, freely licensed assets. Mixamo [28] was selected as the source for the avatar because it provides pre-rigged, pre-textured FBX characters under a license that permits use in non-commercial applications, and because its bone hierarchy (which includes named bones for every major body segment down to individual finger phalanges) maps naturally onto the Plug-in Gait marker set.

The FBX files from Mixamo consist of an internal skeleton ("rig") covered with a mesh, as shown in Figure 2.2. Three.js can support Mixamo's native animations through the `AnimationMixer` class, which interpolates skeleton bone positions over time according to the selected clip data.

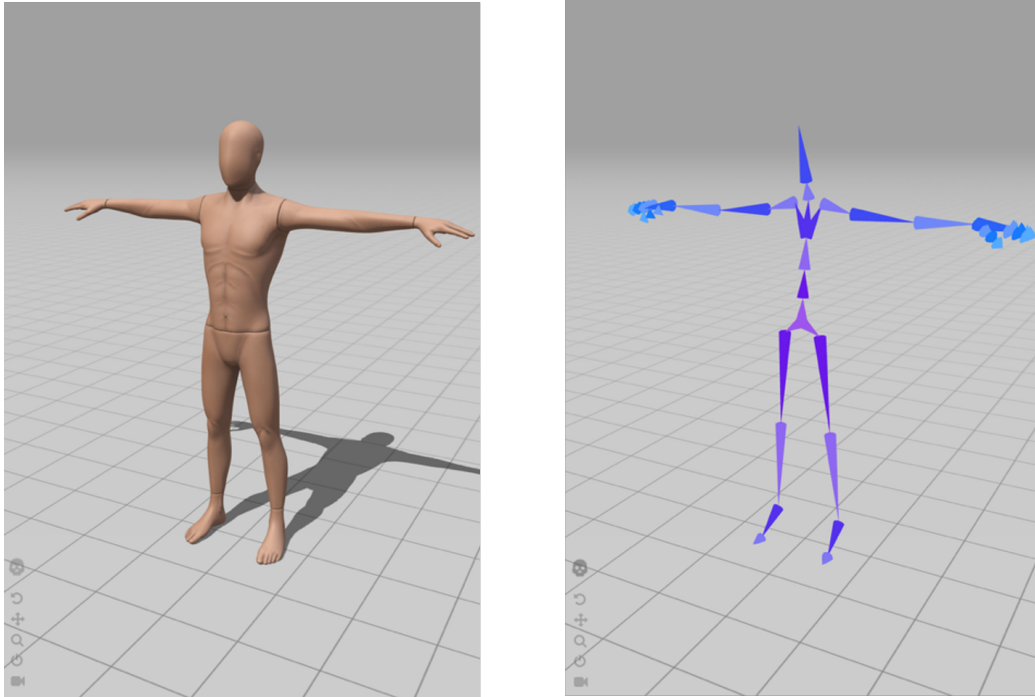


Figure 2.2: Screenshot from Mixamo’s website [28]. Mixamo avatar with and without the mesh, so the rig can be seen.

2.4 Data Visualization

2.4.1 Plotly.js

For the 2D analytical layer, three JavaScript charting libraries were evaluated: Chart.js [29], D3.js [30], and Plotly.js [31]. Chart.js is well-suited for simple, static dashboards but lacks the interactive features (range zoom, pan, hover value inspection) that are essential for clinical signal analysis.

D3.js offers virtually unlimited flexibility but requires constructing every visual element from low-level SVG primitives, implying a significant development overhead for standard time-series charts.

Plotly.js, built on top of D3.js and stack.gl [32], provides the interactivity of a custom D3 implementation through a high-level declarative API, and its `scattergl` trace type leverages WebGL acceleration to handle the high-frequency datasets (1,000 Hz and above) typical of EMG and force-plate recordings. Its `relay` API also enables the real time update of a vertical cursor synchronized to the animation frame.

Analog data visualization will therefore employ Plotly.js, enabling clinicians to zoom into specific gait intervals, pan axes, and query exact values by hovering over signals; providing essential analytical capabilities for clinical interpretation.

2.5 Deployment Infrastructure

2.5.1 Docker Containerization

Deploying a Python application with C++ binary dependencies across heterogeneous clinical hardware is a known challenge. The two main alternatives to containerization, a native installer per operating system or a cloud-hosted SaaS (Software as a Service) model, present drawbacks in a healthcare context: Native installers require per-machine maintenance and are sensitive to OS (Operating System) updates, while cloud deployment raises data governance concerns around the upload of patient movement recordings.

Docker containerization resolves both issues: The application runs identically on any host operating system through a self-contained image, and data processing occurs locally on the institution's own infrastructure. To ensure that CHLOE operates as a portable tool easily deployable in clinical environments without complex configurations, the architecture utilizes Docker containerization [33].

A two-stage build strategy optimizing image size and security was planned for CHLOE:

1. **Build Stage:** Using a Node.js environment to compile the frontend. Tools like Vite can package and reduce JavaScript code and dependencies (Three.js, Plotly.js) into optimized static files.
2. **Server Stage:** Employing a Mambaforge [34] base image (a lightweight Conda [35] distribution) to handle the project's binary dependencies. While backend logic and API were developed using the Flask micro-framework, Gunicorn served as the WSGI application server for deployment.

Requirements

This chapter defines the technical and operational specifications that CHLOE must satisfy to bridge the “clinical gap” identified in the context. These requirements were derived from both technical constraints of the C3D format and usability feedback provided by clinical professionals during the development phase.

3.1 Functional Requirements (FR)

Functional requirements describe the specific behaviors and services that the system must provide.

- **FR1.** *C3D Data Ingestion:* The system shall allow users to upload binary C3D files through a web interface. The backend must be capable of parsing the file regardless of its internal architecture (Intel or DEC).
- **FR2.** *3D Marker Visualization:* The application shall render the 3D point cloud of markers in a virtual environment. Users must be able to toggle the visibility of specific markers or groups (e.g. Pelvis, Lower Limb).
- **FR3.** *Multi-Manufacturer C3D Support:* The system shall be compatible with C3D files generated by different MoCap providers (e.g., Vicon, Motion Analysis Corporation, OptiTrack, etc.). This requires the backend to handle variations in labeling,

coordinate systems, and internal data structures.

- **FR4.** *Biomechanical Avatar Animation:* The system shall map the C3D marker data to a humanoid avatar (Mixamo skeleton). The avatar’s movement must be driven by vectors derived from the markers rather than simple point-to-point attachment to ensure anatomical consistency.
- **FR5.** *Analog Signal Plotting:* The system shall identify and extract analog channels (such as EMG or ground reaction forces) and display them in interactive 2D charts.
- **FR6.** *Temporal Synchronization:* The 3D animation of the avatar and the 2D analog plots must be synchronized. A master timeline shall control both, ensuring that a specific frame in the animation corresponds exactly to the data point in the plot.
- **FR7.** *Multi-language Support:* To facilitate international clinical use, the interface must support dynamic switching between Spanish, English, and French without requiring a page reload.
- **FR8.** *Metadata Extraction:* The system shall display essential trial information, including the sampling rate (FPS, frames per second), the total number of frames, and the original data manufacturer.

3.2 Non-Functional Requirements (NFR)

Non-functional requirements define the quality attributes and constraints of the system.

- **NFR1.** *Portability and Deployment:* The system must be containerized using Docker to ensure that it can be deployed on any server (Windows, Linux, or macOS) without dependency conflicts, specifically handling C++ binary requirements for `ezc3d`.
- **NFR2.** *Performance (Frame Rate):* The 3D visualization must maintain a consistent rendering speed of 60 frames per second (FPS) on standard modern web browsers (in computers, tablets and phones) to ensure smooth movement perception for clinical analysis.
- **NFR3.** *Robustness and Data Integrity:* The backend must handle “corrupt” data or missing marker values (NaN). The system should not crash if a marker is missing in certain frames; instead, it should gracefully hide the marker or use fallback orientation data (e.g., using head markers if pelvis markers are missing).

- **NFR4. Usability:** The interface must be intuitive for non-technical users (physiotherapists). It should follow a “drag-and-drop” philosophy, minimizing the number of clicks required to go from file upload to data visualization.
- **NFR5. Concurrency and Responsiveness:** The local server instance must be capable of handling simultaneous requests from multiple users. Since parsing and extracting data from a C3D file is a CPU-bound operation, the deployment architecture must utilize a WSGI server (Gunicorn) with multiple worker processes, ensuring that a heavy parsing task from one user does not block requests from others.

Table 3.1: Summary of Functional and Non-Functional Requirements.

ID	Requirement description
<i>Functional Requirements (FR)</i>	
FR1	C3D Data Ingestion (architecture independent)
FR2	3D Marker Visualization and visibility toggling
FR3	Multi-Manufacturer Support (Vicon, OptiTrack, etc.)
FR4	Biomechanical Avatar Animation (vector-driven)
FR5	Analog Signal Plotting (EMG / Force)
FR6	Temporal Synchronization (3D Animation / 2D Plots)
FR7	Multi-language Support (ES, EN, FR)
FR8	Extraction and display of trial metadata
<i>Non-Functional Requirements (NFR)</i>	
NFR1	Portability and Deployment via Docker
NFR2	Performance (stable 60 FPS rendering)
NFR3	Robustness against missing data (NaN handling)
NFR4	Usability (intuitive drag-and-drop UI)
NFR5	Concurrency and Responsiveness (Gunicorn/WSGI concurrent requests)

Design and Implementation

This chapter details the design and implementation of CHLOE and describes the translation of clinical and functional requirements into a target software infrastructure. The design focuses on bridging the gap between specialized laboratory data (C3D format) and accessible clinical visualization through web-based technologies.

The implementation is guided by a modular and decoupled philosophy, ensuring that data processing, biomechanical mapping, and user interaction are handled by independent layers. This structure not only meets the portability and performance needs identified in Chapter 3, but also provides a scalable foundation for future extensions. While key code snippets are analyzed in the following sections, CHLOE's complete source code can be found in this GitHub repository: <https://github.com/belengomezmartinez/LibreMotion-CHLOE.git>

4.1 System Architecture

The project is organized into a hierarchical structure in which each component has a specific responsibility within the data ingestion and visualization pipeline.

4.1.1 Directory and File Organization

The source code is divided into backend logic (Python), frontend modules (JavaScript), and static assets, as follows:

- **index.html**: The entry point of the web application. It defines the Document Object Model (DOM) structure, including the 3D canvas container, the control sidebars, and the dynamic panels for markers and telemetry.
- **styles.css**: Contains the global style definitions. It implements the responsive grid layout, the Dark/Light mode color variables, and the specific CSS transitions for the interactive UI (User Interface) components.
- **processor.py**: The biomechanical engine of the system. It handles the binary parsing of C3D files, implements coordinate system transformations (Z-up to Y-up) and applies clinical data standardization through predefined marker maps, see Section 4.2.3.
- **app.py**: A Flask-based REST (Representational State Transfer) API (Application Programming Interface) that serves as the communication bridge. It manages secure file uploads via temporary storage and orchestrates the exchange of processed biomechanical data between the Python engine and the web client.
- **src/**: This directory contains the modularized JavaScript logic responsible for user interaction and rendering:
 - **main.js**: Management of the Three.js scene, the application's global state, and the main animation loop.
 - **kinematics.js**: Segmental alignment logic and real-time skeletal positioning of the humanoid avatar.
 - **vectors.js & plots.js**: Calculation of biomechanical angles and synchronization of the real-time telemetry displayed on the Plotly dashboards.
 - **trajectories.js**: Generation and rendering of 3D movement paths for specific markers.
 - **i18n.js**: An internationalization engine that provides localized support for Spanish, English, and French.
 - **constants.js**: Storage of the biomechanical metadata, marker categories, and color palettes used throughout the application.

- **public/maniqui.fbx**: A 3D humanoid model that serves as an anatomical avatar. Its internal bone hierarchy is mapped by the software to reflect the user's locomotion data.

4.1.2 Environment Configuration and Deployment

CHLOE is designed to be a portable and cross-platform tool. This is achieved through specific configuration and containerization files:

- **Dockerfile**: Definition of the container image, ensuring that the application, its Python environment, and the necessary C++ dependencies for `ezc3d` are consistently deployed, regardless of the host OS.
- **package.json**: Frontend dependency management, specifically Three.js and Plotly.js.
- **.env**: Storage of environment-specific variables following industry standards for secure software decoupling. In CHLOE, this file centralizes critical runtime configurations, specifically defining `FLASK_PORT=8080` to route HTTP traffic without port conflicts, `FLASK_WORKERS=4` to provision four isolated parallel processes, ensuring non-blocking frontend requests during CPU-bound data parsing, and `FLASK_TIMEOUT=120` to extend Gunicorn's execution window to 120 seconds, preventing worker termination during the ingestion of heavy or high-frequency C3D trials.

One of the project's technical challenges proved to be installing the `ezc3d` library, which requires the compilation of C++ code on the author's computer, a MacBook with an ARM processor. Installation via `pip` frequently failed on modern systems due to a lack of compatible precompiled binaries. The solution was to use Mamba (a faster package manager than Conda [35]) to install `ezc3d`, `numpy`, `flask` and `gunicorn` from the `conda-forge` repository. This approach downloads precompiled and optimized binaries, ensuring that the container functions identically on any server, regardless of the underlying hardware.

4.2 Data Ingestion and Processing Pipeline

The data ingestion pipeline is the stage in which raw biomechanical measurements are transformed into a standardized format for the web interface. This process involves secure file transmission, binary parsing, and anatomical data normalization.

4.2.1 Asynchronous File Ingestion and Synchronization

The communication between the client and the server follows a RESTful architecture. As illustrated in Figure 4.1, the process is triggered by a user interaction, where the file is

transmitted to the Flask backend through a form-data POST request.

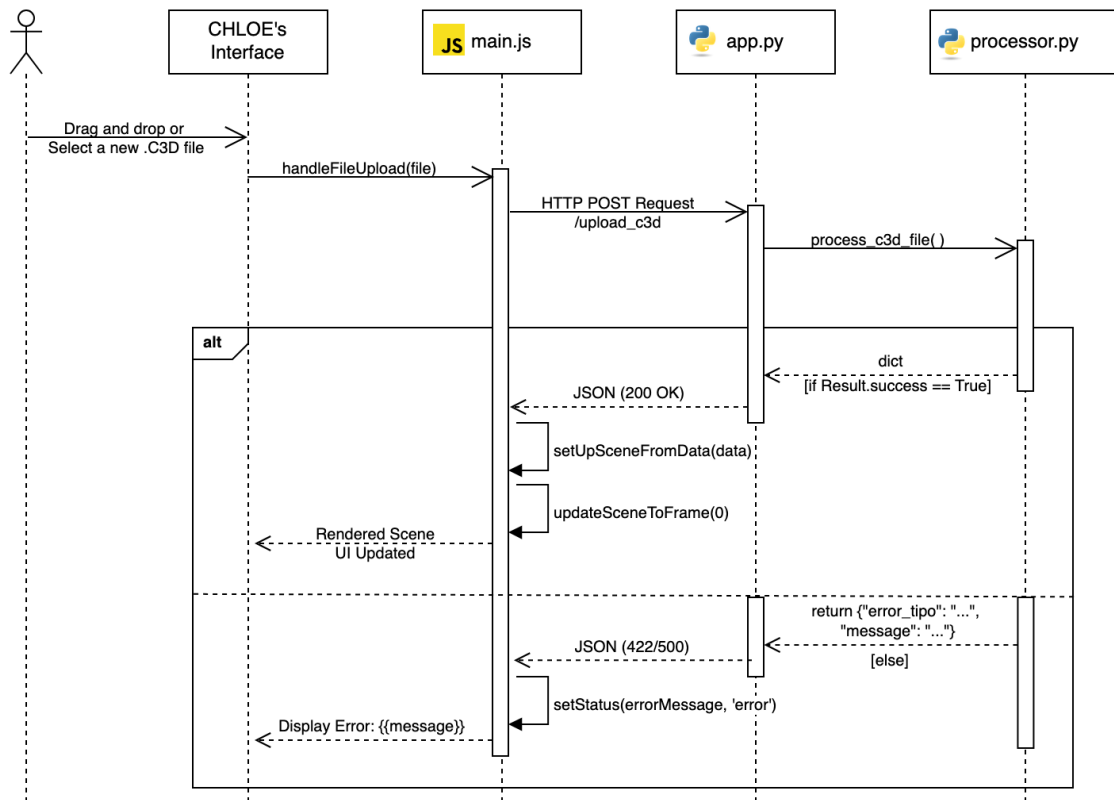


Figure 4.1: Sequence diagram of the C3D ingestion pipeline and error handling logic.

The system implements a error-handling mechanism modeled via an `alt` fragment. If the file is successfully parsed, the server returns a `200 OK` status with the processed JSON data. However, if the file violates C3D standards or uses unsupported architectures, the backend propagates a `422 error`, which the frontend captures to notify the user via a localized status message.

4.2.2 Exception Handling and Data Integrity

The system implements an additional security layer within `app.py` to ensure data integrity during the serialization phase. Even if the `processor.py` engine completes the parsing without exceptions, the API performs a final validation before transmission.

A `try-except` block specifically catches `ValueErrors` related to numerical inconsistencies, such as `NaN` (Not a Number) or `Infinity` values that could crash the frontend's rendering engine. In such cases, the server raises a `500 Internal Server Error` with a specific message, preventing the transmission of corrupted telemetry and maintaining the stability of the WebGL context.

```

app.py

# Controlled errors (returned by processor.py)
if "error_tipo" in result:
    return jsonify({"error": result['message'], "tipo":
        ↪ result['error_tipo']}), 422

# Data integrity validation (Numerical errors)
if result.get("success"):
    try:
        return jsonify(result)
    except ValueError:
        return jsonify({"error": "Numerical error (NaN) in the processed
            ↪ file"}), 500

```

4.2.3 Biomechanical Data Standardization

Once the file reaches the `processor.py` engine, it undergoes several transformation stages using the `ezc3d` library:

- **Unit Scaling:** Laboratory data can be recorded even in millimeters, centimeters (although this is rarer), or meters. Through `scalar()`, the system attempts to use the parameter `UNIT` from the C3D file, but if it is empty or does not provide useful information, it defaults to an automatic detection algorithm that analyzes the coordinate range to ensure that all outputs are standardized to the metric system.
- **Coordinate System Transformation:** Clinical data typically follows a Z-up (Laboratory) orientation. `clean_marker_data()` transforms each marker coordinate from $[X, Y, Z]$ to $[X, Z, -Y]$, ensuring that the Y-axis represents the vertical height in the virtual scene.
- **Marker Mapping:** The engine utilizes a `STANDARD_MAP` to normalize heterogeneous marker labels into a consistent clinical nomenclature (e.g., converting “L_ASIS” to “LAST”). See Section 4.4.1.

4.2.4 Analog Signal Classification

Parallel to marker data, the system parses analog channels (EMG, force plates, etc.). A keyword-based classification routine, `classify_analog()`, analyzes channel labels against clinical keywords to automatically assign user-interface colors and categories, facilitating data interpretation for the clinician.

4.2.5 Data Orchestration and Output Structure

The final stage of the backend pipeline is managed by the `process_c3d_file()` function, which serves as the main orchestrator. This function integrates binary parsing, file integrity checks, and metadata extraction to fulfill several functional requirements of the system (FR1, FR3, FR8).

As part of this process, the engine invokes `extract_analog_data()`, a specialized sub-routine that parses, classifies, and sanitizes high-frequency analog datasets to ensure they are ready for clinical visualization. The result is a structured dictionary (dict) returned to the Flask wrapper with the following schema:

```
processor.py
# process_c3d_file(c3d_file_path):
return {
    "success": True,
    "frames": frames,           # A list of dictionaries (marker:coord)
    "original_fps": fps,       # Data acquisition frequency
    "duration_seconds": duration, # Total trial length
    "total_frames": n_frames,
    "has_analog_data": analog is not None,
    "analog_data": analog      # Categorized clinical signals
}
```

The `frames` and `analog_data` fields represent the core biomechanical payload of the system. Their internal structure is optimized for $O(1)$ retrieval during the 3D rendering and telemetry phases:

- **frames:** A list where each element represents a discrete moment in time. Each entry is a dictionary where the keys are standardized marker labels (e.g., “LAST”, “RHEE”) and the values are coordinate triplets $[X, Z, -Y]$. By mapping these labels once during the ETL (Extract, Transform and Load) phase, the frontend can perform $O(1)$ lookups to update the 3D spheres without additional string parsing.
- **analog_data:** Structure optimized as a dual-map scheme for efficient frontend telemetry retrieval:
 - `channels:` A dictionary indexed by sanitized channel labels (e.g., “EMG 1”, “Force 1”) storing flat arrays of numerical samples. Out-of-bound values (NaN/infinities) are replaced with null elements to prevent client-side serialization crashes.
 - `classification:` A parallel lookup table pairing each channel with its clinical metadata computed by `classify_analog()` (the signal’s `tipo` derived from the `STANDARD_ANALOG_MAP`, a hex color code for UI synchronization, and its original physical index).

- Global Configuration Metadata: Root fields containing the converter sampling frequency (`sample_rate_hz`) and the active clinical groups (`tipos_disponibles`), enabling the Plotly.js dashboard to dynamically instantiate chart filters and tabs.

This decoupling of visual (frames) and analytical (analog) data allows CHLOE to maintain a stable 60 FPS animation loop while asynchronously updating complex Plotly graphs only when necessary.

4.3 Real-time Visualization Engine

The visualization engine is responsible for synchronizing the high-frequency biomechanical data with the graphical representation in the browser. This requires a stable execution loop that decouples the data acquisition rate from the display's refresh rate.

4.3.1 System Initialization

Before the real-time loop begins, the system executes `init()` once. This function acts as the bootstrap for the frontend, ensuring that all dependencies and UI components are ready. The initialization process generates the default user interface layout (see Figure 4.2).

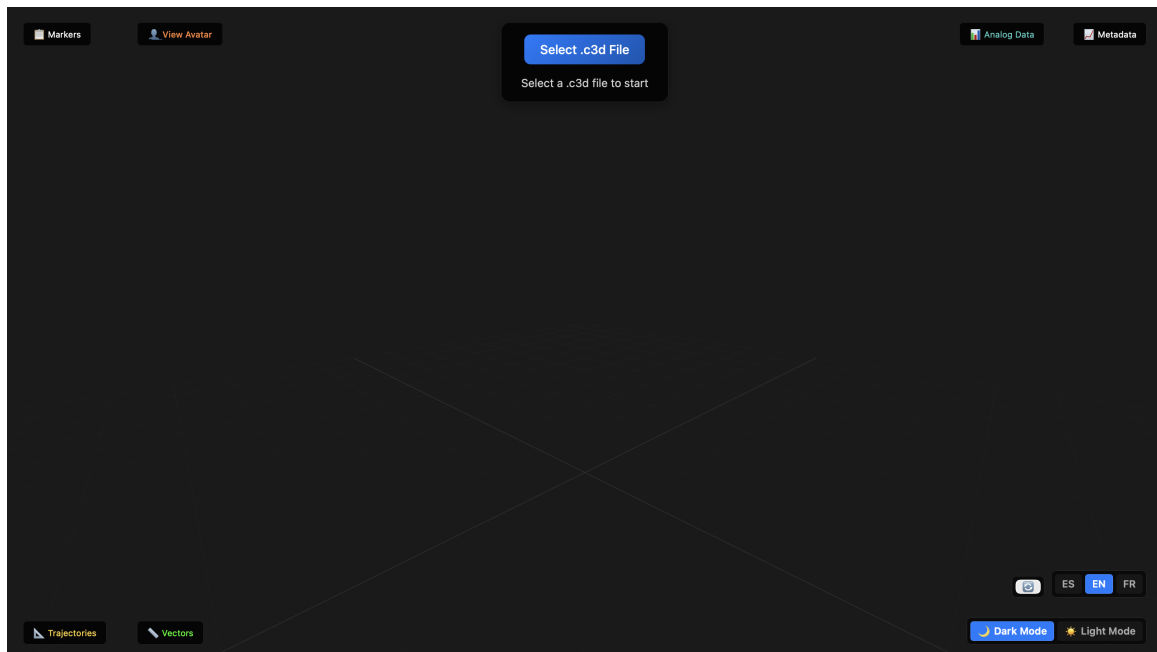


Figure 4.2: CHLOE main interface initialization with a 3D viewport and control sidebars.

As illustrated in the sequence diagram (Figure 4.3), `init()` performs the following tasks:

- **Avatar Loading:** Invokes `loadAvatar()` to import the FBX humanoid model and set up the skeletal hierarchy.
- **Localization:** Calls `changeLanguage('es')` to initialize the `i18n` engine and translate the UI labels.
- **Event Listeners:** Sets up the Theme Toggle and other interactive controls via `initThemeToggle()`.
- **Loop Triggering:** Finally, it makes the first call to `animate()`, which then utilizes `requestAnimationFrame()` to maintain the continuous rendering cycle.

4.3.2 The Animation Loop and Temporal Synchronization

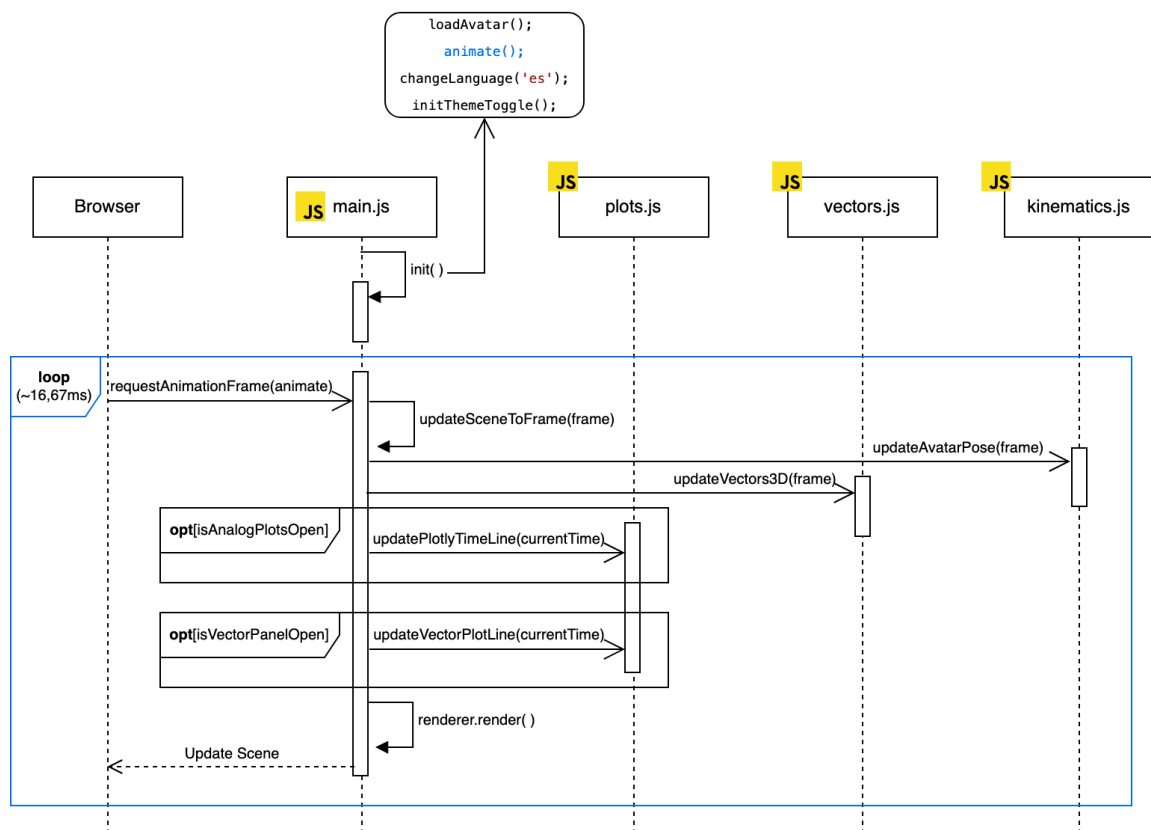


Figure 4.3: Sequence diagram of the real-time animation loop and telemetry synchronization.

As shown in Figure 4.3, the system is driven by the `requestAnimationFrame` API. This ensures that the `animate()` function is called in sync with the monitor's refresh rate, typically every 16.67ms (60 Hz).

To ensure that the animation plays at the intended playback speed regardless of hardware performance, a *delta-time* (Δt) strategy is implemented. This logic is encapsulated in the following code snippet from `main.js`:

```

// main.js

// animate()
// Calculate time elapsed since last frame
const deltaTime = lastTime ? (currentTime - lastTime) / 1000 : 0.016;
lastTime = currentTime;

if (isPlaying && animationData) {
  frameAccumulator += deltaTime * playbackSpeed;
  const frameTime = 1 / originalFPS;

  const framesToAdvance = Math.floor(frameAccumulator / frameTime);
  if (framesToAdvance > 0) {
    frameIndex = (frameIndex + framesToAdvance) % animationData.length;
    frameAccumulator %= frameTime;
    updateSceneToFrame(frameIndex);
  }
}

```

The `frameAccumulator` allows the handling of datasets with different acquisition frequencies (e.g., 100 Hz or 200 Hz) while maintaining smooth visual playback on a standard 60 Hz screen.

4.3.3 Conditional Updates

A key optimization in CHLOE is the conditional execution of non-essential updates: Computationally intensive operations like Plotly's layout for analog signals and vector plot updates are only triggered if their respective UI panels are open (`isAnalogPlotsOpen` or `isVectorPanelOpen`). This ensures that the main thread remains free for 3D rendering, preventing UI lag during the playback of large clinical trials (see `opt` in Figure 4.3).

4.3.4 Continuous State Synchronization

The animation loop triggers two update functions every frame regardless of whether the telemetry panels are visible: `updateVectors3D()` and `updateAvatarPose()` (see Sections 4.5.2 and 4.4.3, respectively), which drive avatar mode bone orientations.

4.4 Kinematic Mapping and Avatar Animation

A significant task in biomechanical visualization is the translation of sparse marker coordinates into a continuous skeletal posture. CHLOE addresses this through a two-stage mapping process that ensures compatibility between clinical standards and graphics assets. The engine supports two user-selectable rendering modes: **Skeleton mode**, which represents the subject as a wireframe figure, and **avatar mode**, which maps the data onto a 3D model. Figure 4.4 illustrates both visualization modes.

The skeleton mode provides a direct representation of marker connectivity, while the avatar mode maps the same data onto a 3D humanoid model, offering a direct visual context of movement. They are described in Sections 4.4.2 and 4.4.3, respectively.



Figure 4.4: Comparison between skeleton mode (a) and avatar mode (b) for the same frame of motion capture data.

4.4.1 Standardization via Plug-in-Gait Protocol

To ensure the system is agnostic to the motion capture hardware, the `processor.py` engine first transforms the heterogeneous raw labels into a `STANDARD_MAP`. This map follows the Plug-in-Gait clinical protocol, the most widely used convention in gait analysis.

By normalizing markers to labels (`LASI`, `RASI`, ...), the frontend can rely on a consistent anatomical nomenclature regardless of the original file's manufacturer. This normalization is shown in Table 4.1. It is important to note that although the `RBAK` marker belongs to the Plug-in-Gait protocol, it was not added to the `STANDARD_MAP` because it does not provide any biomechanical information; it is only used to differentiate the right side from the left. Finally, `STANDARD_MAP` hand markers belonging to the LibreMotion Project were also added. They are shown in Table 4.2.

In the marker control panel, the user interface maps and displays the anatomical nomenclature defined by the `anatomical_names` property within the `TRANSLATIONS` constant, followed by the specific Plug-in Gait (PiG) label enclosed in parentheses.

Table 4.1: Standard marker nomenclature equivalences across motion capture manufacturers. The Plug-in Gait (Vicon Nexus) label is the reference standard used throughout CHLOE. Markers marked with * do not appear in the official Plug-in Gait documentation.

Anatomical landmark	PiG				
7th cervical vertebra	C7	C7_SPINE			
10th thoracic vertebra (posterior)	T10	T10_SPINE			
Clavicle (suprasternal notch)	CLAV	CLAVICLE			
Sternum (xiphoid process)	STRN	STERNUM			
Left ant. superior iliac spine	LASI	LFWT	LAntInflIiacSpine	L.ASIS	L_ASIS
Right ant. superior iliac spine	RASI	RFWT	RAntInflIiacSpine	R.ASIS	R_ASIS
Left post. superior iliac spine	LPSI	LBWT	LPostInflIiacSpine	L.PSIS	L_P SIS
Right post. superior iliac spine	RPSI	RBWT	RPostInflIiacSpine	R.PSIS	R_P SIS
Sacrum (posterior pelvis)	SACR	SACRAL	VSAC		
Left shoulder (acromion)	LSHO	L_SHOULDER			
Left upper arm (lateral)	LUPA	L_UPPER_ARM			
Left lateral epicondyle (elbow)	LELB	L_ELBOW			
Left forearm	LFRM	L_FOREARM			
Left wrist ulnar styloid	LWRB				
Left wrist (midpoint)*	LWRT*	L_WRIST	LWRIST	LWRI	
Left wrist radial styloid	LWRA				
Right shoulder (acromion)	RSHO	R_SHOULDER			
Right upper arm (lateral)	RUPA	R_UPPER_ARM			
Right lateral epicondyle (elbow)	RELB	R_ELBOW			
Right forearm	RFRM	R_FOREARM			
Right wrist ulnar styloid	RWRB				
Right wrist (midpoint)*	RWRT*	R_WRIST	RWRIST	RWRI	
Right wrist radial styloid	RWRA				
Left thigh (lateral wand)	LTHI	L_THIGH	L.Ant.Fem.		

(continues on next page)

(continued)

Anatomical landmark	PiG					
Left lat. femoral epicondyle (knee)	LKNE	L_KNEE	LLFC	L.Knee		
Left shin (tibial wand)	LSHN	L_SHIN	LTIB	LSHA	LTTB	L.Ant.Tib.
Left lateral malleolus (ankle)	LANK	L_ANKLE	LLMA	L.Ankle		
Left heel (calcaneus)	LHEE	L_HEEL	LCAL	L.Heel		
Left 1st metatarsal head	LTOE	L_TOE	L.TO	L.Toe		
Left 5th metatarsal head	LMT5	L_FOOT				
Right thigh (lateral wand)	RTHI	R_THIGH	R.Ant.Fem.			
Right lat. femoral epicondyle (knee)	RKNE	R_KNEE	RLFC	R.Knee		
Right shin (tibial wand)	RSHN	R_SHIN	RTIB	RSHA	RTTB	R.Ant.Tib.
Right lateral malleolus (ankle)	RANK	R_ANKLE	RLMA	R.Ankle		
Right heel	RHEE	R_HEEL	RCAL	R.Heel		
Right 1st metatarsal head	RTOE	R_TOE	R.TO	R.Toe		
Right 5th metatarsal head	RMT5	R_FOOT				
Right back head	RBHD	R_BACK_HEAD				
Left back head	LBHD	L_BACK_HEAD				
Left front head	LFHD	L_FRONT_HEAD				
Right front head	RFHD	R_FRONT_HEAD				

* LWRT and RWRT are not part of Plug-in Gait. They represent the virtual midpoint between the radial and ulnar styloid markers (LWRA/LWRB and RWRA/RWRB), computed dynamically by CHLOE as the forearm bone anchor (see Section 4.4.3.3).

Table 4.2: Hand and finger marker set used in CHLOE, following the LibreMotion naming convention. Each marker label encodes laterality (R/L), finger identity, and phalange level.

LibreMotion		LibreMotion	
R marker		L marker	
<i>Palm</i>			
RPLM	RPALM	LPLM	LPALM
<i>Thumb</i>			
RTMC	RTHUMB_METACARPAL	LTMC	LTHUMB_METACARPAL
RTPX	RTHUMB_PROXIMAL	LTPX	LTHUMB_PROXIMAL
RTDI	RTHUMB_DISTAL	LTDI	LTHUMB_DISTAL

(continues on next page)

(continued)

R marker		L marker	
RTTP	RTHUMB_TIP	LTP	LTHUMB_TIP
Index			
RIMC	RINDEX_METACARPAL	LIMC	LINDEX_METACARPAL
RFIN	RINDEX_PROXIMAL	LFIN	LINDEX_PROXIMAL
RIIM	RINDEX_INTERMEDIATE	LIIM	LINDEX_INTERMEDIATE
RIDI	RINDEX_DISTAL	LIDI	LINDEX_DISTAL
RITP	RINDEX_TIP	LITP	LINDEX_TIP
Middle			
RMMC	RMIDDLE_METACARPAL	LMMC	LMIDDLE_METACARPAL
RMPX	RMIDDLE_PROXIMAL	LMPX	LMIDDLE_PROXIMAL
RMIM	RMIDDLE_INTERMEDIATE	LMIM	LMIDDLE_INTERMEDIATE
RMDI	RMIDDLE_DISTAL	LMDI	LMIDDLE_DISTAL
RMTP	RMIDDLE_TIP	LMTP	LMIDDLE_TIP
Ring			
RRMC	RRING_METACARPAL	LRMC	LRING_METACARPAL
RRPX	RRING_PROXIMAL	LRPX	LRING_PROXIMAL
RRIM	RRING_INTERMEDIATE	LRIM	LRING_INTERMEDIATE
RRDI	RRING_DISTAL	LRDI	LRING_DISTAL
RRTP	RRING_TIP	LRTP	LRING_TIP
Little (pinky)			
RLMC	RLITTLE_METACARPAL	LLMC	LLITTLE_METACARPAL
RLPX	RLITTLE_PROXIMAL	LLPX	LLITTLE_PROXIMAL
RLIM	RLITTLE_INTERMEDIATE	LLIM	LLITTLE_INTERMEDIATE
RLDI	RLITTLE_DISTAL	LLDI	LLITTLE_DISTAL
RLTP	RLITTLE_TIP	LLTP	LLITTLE_TIP

4.4.2 Skeleton Mode

This is the default rendering mode upon file load. When this wireframe representation is active, the subject is rendered as a skeletal figure built entirely from the standardized markers shown in Section 4.4.1. During `init()`, shared geometries and materials are pre-allocated once into `scene.userData`:

- `THREE.SphereGeometry`: Small opaque spheres for the markers and a big semi-transparent sphere for the head.
- `THREE.LineBasicMaterial`: Lines for the bone segments.

When a file is loaded, `setUpSceneFromData()` iterates over the `CONNECTIONS` object (an ordered list of standardized marker-name pairs representing anatomically adjacent landmarks) and, for each valid pair, creates a `THREE.Line` with a pre-allocated `Float32Array` position buffer, whose two endpoints are updated every frame to the current marker world-space positions. If the visibility of either defining marker endpoint is deactivated by the user via the interface marker panel, the corresponding `THREE.Line` object undergoes a visibility state update, automatically hiding the segment line from the 3D viewport to ensure scene consistency. Markers not present in the `STANDARD_MAP` are still rendered as gray spheres but are excluded from `CONNECTIONS` and therefore appear unconnected.

When switching to Avatar mode, marker spheres remain visible in the scene; only the `CONNECTIONS` lines are hidden, so the clinician retains spatial reference to the raw capture data while the avatar animation is active.

4.4.3 Avatar Mode

As explained in Section 4.2.5, each frame of captured motion data provides a dictionary of named 3D positions (`marker:coord`); the `updateAvatarPose()` function, implemented in `kinematics.js`, consumes these positions and drives the corresponding avatar bones through a hierarchical position computation strategy:

Global root placement → Global orientation → Segment-by-segment bone alignment.

The core mathematical primitive is the `alignBone()` helper, which reorients a Mixamo skeleton bone so that its local Y axis points from a given origin marker to a given target marker. The Mixamo FBX model is traversed at load time to extract bone references into the `avatarBones` object.

4.4.3.1 Bone Alignment Algorithm (`alignBone`)

All bone rotations are computed by the same five-step quaternion procedure:

```
kinematics.js
function alignBone(bone, pointStart, pointEnd) {
  // 1. Compute world direction vector
  const targetDir = new THREE.Vector3().subVectors(pointEnd,
    ↪ pointStart).normalize();

  // 2. Get parent world rotation to isolate local space
  const parentRotation = new THREE.Quaternion();
  if (bone.parent) {
    bone.parent.getWorldQuaternion(parentRotation);
  }
}
```

```

    }

    // 3. Transform target direction to local space using the inverse rotation
    const invParentRotation = parentRotation.clone().invert();
    const localTargetDir =
    ↪ targetDir.clone().applyQuaternion(invParentRotation);

    // 4. Solve rotation from the default 'Up' vector (0,1,0) to the local
    ↪ target
    const quaternion = new THREE.Quaternion();
    quaternion.setFromUnitVectors(new THREE.Vector3(0, 1, 0), localTargetDir);

    // 5. Apply the resulting quaternion to the bone
    bone.quaternion.copy(quaternion);
}

```

Using the inverse of the parent world quaternion ensures that the computed rotation is expressed in the bone’s local coordinate frame, preserving the integrity of the hierarchical skeleton chain regardless of the parent’s accumulated rotation.

4.4.3.2 Root Placement and Global Orientation

Before any bone is rotated, the avatar mesh is repositioned and reoriented in world space:

- **Position (XZ plane):** Centroid of all available pelvis markers (SACR, RASI, LASI, RPSI, and LPSI). If none are present, the head centroid is used, and 0.80 m is subtracted along the Y axis to approximate the pelvis height.
- **Position (Y / grounding):** The avatar’s vertical origin is set to $(\text{pelvisPos.y} - \text{avatarRestHipHeight})$, where `avatarRestHipHeight` is the world-space Y coordinate of the Hips bone measured once at load time from the FBX bind pose. This anchors the model’s hip joint to the subject’s real pelvis height regardless of the activity, including seated tasks where foot-based grounding would fail.
- **Global rotation:** the forward-facing direction is the cross product of the hip axis vector ($\text{LASI} - \text{RASI}$) with the world up vector $(0, 1, 0)$, with fallbacks cascading through the shoulder axis and the forehead axis.

4.4.3.3 Kinematic Chain: Marker Pairs per Bone

Once the root is placed, each body segment is driven independently by the marker pairs listed in Table 4.3.

Table 4.3: Kinematic chain: Origin marker $\rightarrow$ Target marker $\rightarrow$ Mixamo bone driven.

Origin marker	$\rightarrow$	Target marker	Mixamo bone driven
<i>Spine</i>			
SACR	$\rightarrow$	(C7+CLAV)/2	Spine · Spine1 · Spine2
<i>Right Upper Limb</i>			
RSHO	$\rightarrow$	RELB	RightArm
RELB	$\rightarrow$	RFRM / <i>wrist</i> ^a	RightForeArm
<i>wrist</i> ^a	$\rightarrow$	RPLM / RFIN	RightHand
marker[i]	$\rightarrow$	marker[i + 1]	RightFingers ($\times 5 \times 3$ phalanges) ^b
<i>Left Upper Limb</i>			
LSHO	$\rightarrow$	LELB	LeftArm
LELB	$\rightarrow$	LFRM / <i>wrist</i> ^a	LeftForeArm
<i>wrist</i> ^a	$\rightarrow$	LPLM / LFIN	LeftHand
marker[i]	$\rightarrow$	marker[i + 1]	LeftFingers ($\times 5 \times 3$ phalanges) ^b
<i>Right Lower Limb</i>			
<i>hip_world</i> ^c	$\rightarrow$	RKNE	RightUpLeg
RKNE	$\rightarrow$	RANK	RightLeg
RANK + 0.05y ^d	$\rightarrow$	RTOE / RMT5	RightFoot
RHEE	$\rightarrow$	RTOE	RightToeBase
<i>Left Lower Limb</i>			
<i>hip_world</i> ^c	$\rightarrow$	LKNE	LeftUpLeg
LKNE	$\rightarrow$	LANK	LeftLeg
LANK + 0.05y ^d	$\rightarrow$	LTOE / LMT5	LeftFoot
LHEE	$\rightarrow$	LTOE	LeftToeBase

(a) Dynamic Wrist Midpoint.

(b) Extremity Sub-routines.

(c) Global Pelvic Anchor.

(d) Ankle-to-Toe Vertical Offset.

Four segment-specific strategies are implemented for skeletal alignment:

Head Gaze: Instead of `alignBone()`, a “look-at” transform is applied. The forward direction is computed as the vector from the midpoint of the back-head markers (RBHD, LBHD) to the midpoint of the forehead markers (RFHD, LFHD).

Dynamic Wrist Midpoint (a): The `ForeArm` bone target is the midpoint of `RWRA` and `RWRB` (or `LWRA/LWRB`), ensuring the segment is centered at the anatomical wrist joint and preventing mesh distortion during pronation and supination.

Table 4.4: FINGER_MARKERS mapping: each phalange pair and its corresponding Mixamo bone.

Finger	Phal. 1 (Prox.)	Phal. 2 (Mid.)	Phal. 3 (Dist.)	Mixamo bones driven
<i>Right hand</i>				
Thumb	RTMC → RTPX	RTPX → RTDI	RTDI → RTTP	RightHandThumb1 · 2 · 3
Index	RIMC → RFIN	RFIN → RIIM	RIIM → RITP	RightHandIndex1 · 2 · 3
Middle	RMMC → RMPX	RMPX → RMIM	RMIM → RMTP	RightHandMiddle1 · 2 · 3
Ring	RRMC → RRPX	RRPX → RRIM	RRIM → RRTP	RightHandRing1 · 2 · 3
Pinky	RLMC → RLPX	RLPX → RLIM	RLIM → RLTP	RightHandPinky1 · 2 · 3
<i>Left hand</i>				
Thumb	LTMC → LTPX	LTPX → LTDI	LTDI → LTTP	LeftHandThumb1 · 2 · 3
Index	LIMC → LFIN	LFIN → LIIM	LIIM → LITP	LeftHandIndex1 · 2 · 3
Middle	LMMC → LMPX	LMPX → LMIM	LMIM → LMTP	LeftHandMiddle1 · 2 · 3
Ring	LRMC → LRPX	LRPX → LRIM	LRIM → LRTP	LeftHandRing1 · 2 · 3
Pinky	LLMC → LLPX	LLPX → LLIM	LLIM → LLTP	LeftHandPinky1 · 2 · 3

Extremity Subroutines (b): Finger kinematics are handled separately by the function `updateHandFingers()`. `FINGER_MARKERS` defines, for each of the five fingers and each hand, an ordered array of four marker names. Three consecutive pairs (index $i \rightarrow i+1$) drive the three phalange bones of that finger via `alignBone()`. Table 4.4 lists all markers used per finger per hand. A special case handles scenes where arm markers are absent: the palm marker (RPLM / LPLM) back-propagates an estimated orientation up through the forearm and upper arm bones, preventing a T-pose. When only wrist data are available, the hand bone’s local position is overridden directly via a world-to-local transform.

Global Pelvic Anchor, `hip_world` (c): The proximal anchor for each thigh is retrieved via `getWorldPosition()` on the `RightUpLeg/LeftUpLeg` bones after the root placement (Section 4.4.3.2), guaranteeing correct joint center alignment regardless of the activity.

Ankle-to-Toe Vertical Offset (+0.05m) (d): A vertical correction is added to the ankle coordinate before the `Foot alignBone()` call to compensate for the Mixamo rig’s default plantar flexion, keeping the sole parallel to the ground during the stance phase.

4.5 Analytical and Visualization Features

Beyond skeletal animation, CHLOE also provides a suite of specialized analytical tools that allow clinicians and researchers to analyze spatial kinematics and high-frequency physiological signals within a unified, synchronized environment.

4.5.1 Dynamic Marker Trajectories

To support the study of displacement patterns (like the vertical oscillation of the center of mass or the swing arc of the foot) the system implements a dedicated trajectory module (`trajectories.js`).

Upon marker selection, the engine iterates over the full animation dataset and populates a `THREE.BufferGeometry` position buffer with the marker's world-space coordinates across all frames. The resulting path is rendered as a persistent `THREE.Line` that remains visible during playback, independently of the current frame. GPU (Graphics Processing Unit) memory is released via `geometry.dispose()` and `material.dispose()` when a trajectory is removed, preventing resource leaks during long sessions.

This allows clinicians to identify gait irregularities (asymmetrical step lengths, abnormal mediolateral deviations, compensatory trunk strategies, etc.) by visually tracing the spatial path of anatomical landmarks such as LHEE or C7. As shown in Figure 4.5, the trajectories panel allows the user to choose the visualization range.

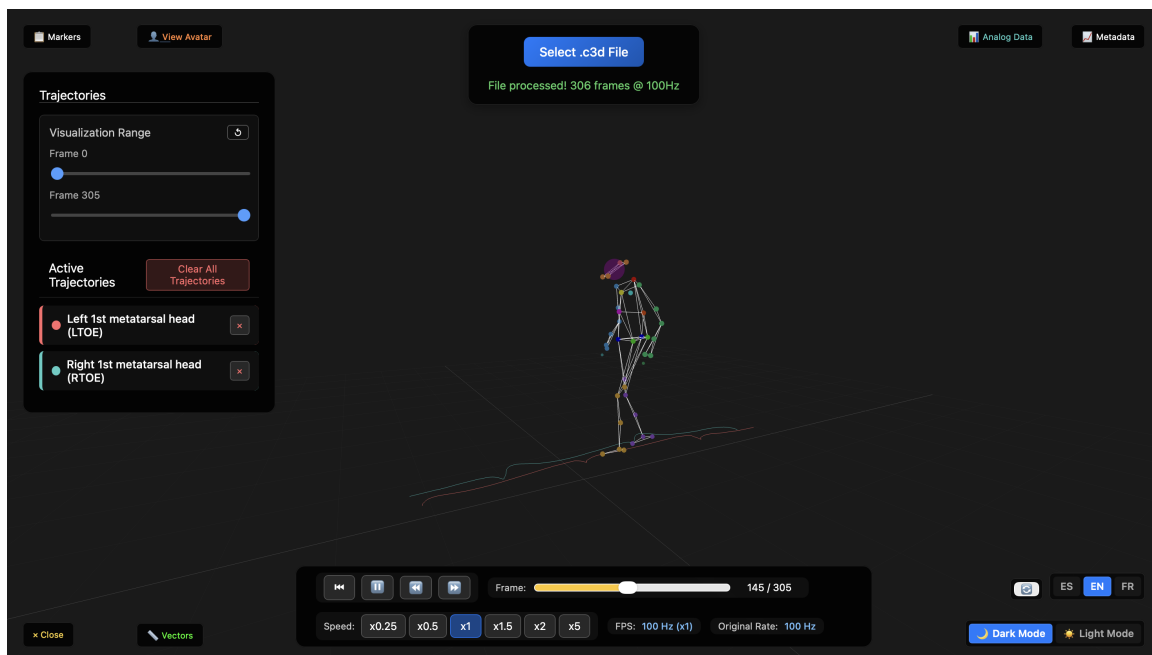


Figure 4.5: 3D visualization of motion capture markers and their trajectories over time.

4.5.2 Interactive 3D Vectors and Angular Analysis

The system includes a real-time geometric analysis module (`vectors.js`) that allows users to define custom inter-marker vectors directly within the 3D scene. Given a source marker A and a target marker B , the instantaneous vector $\vec{v} = P_B - P_A$ is computed at every frame and rendered as a `THREE.ArrowHelper`, whose origin, direction, and length are updated continuously. A local `THREE.AxesHelper` co-located at the vector's origin provides an immediate spatial reference. The engine also pre-computes, for each frame, the signed projection angles of $\vec{v}$ onto the three cardinal planes:

$$\theta_x = \text{atan2}(v_z, v_y), \quad \theta_y = \text{atan2}(v_z, v_x), \quad \theta_z = \text{atan2}(v_y, v_x) \quad (4.1)$$

where θ_x represents the angle of the vector projected onto the YZ plane, θ_y onto the XZ plane, and θ_z onto the XY plane. Unlike the dot-product formulation, `atan2` returns a signed angle in the range $(-180^\circ, 180^\circ]$, preserving directional information and making it sensitive to the side to which a segment deviates.

The resulting time series are displayed as synchronized line charts via `Plotly.js`, with a vertical cursor that tracks the current animation frame, as it is shown in Figure 4.6. This is particularly useful for quantifying trunk lean, limb segment inclination relative to the gravity vector, or the elevation angle of the upper extremity throughout a reaching task.

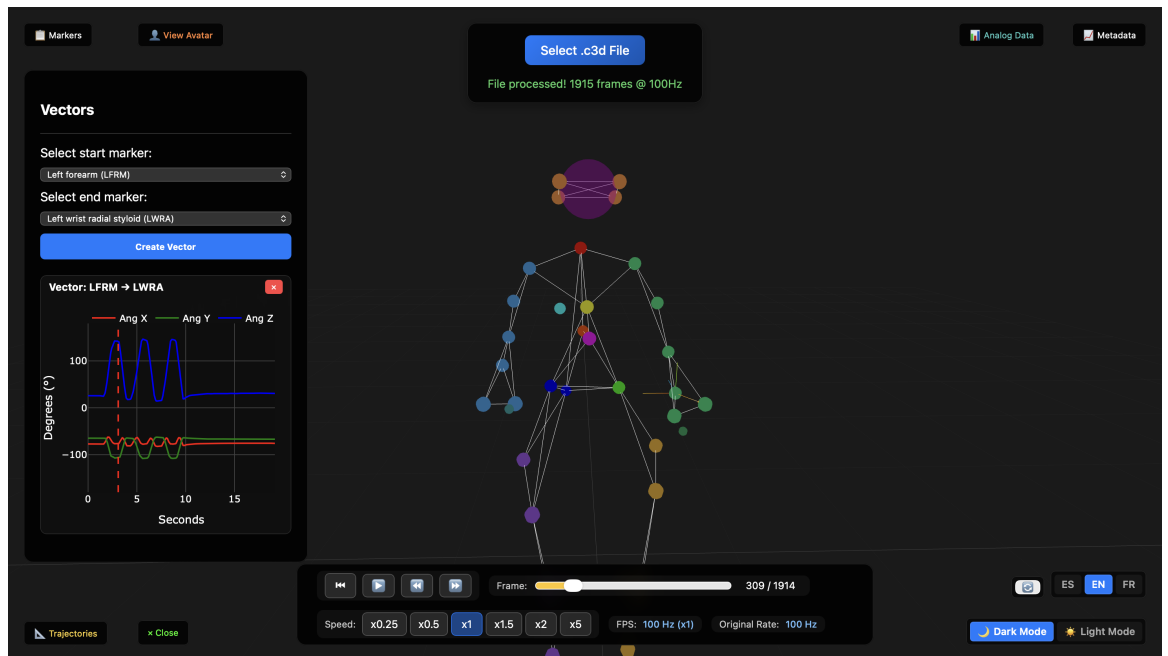


Figure 4.6: Vector-based analysis between the left forearm and left wrist markers, including the decomposition of the vector into its Cartesian projections for quantitative motion assessment.

4.5.3 Analog Signal Visualization

The integration of high-frequency physiological data (like surface electromyography and force-plate recordings) is handled through a synchronized telemetry dashboard implemented in `plots.js`.

These signals are rendered as WebGL-accelerated line charts (`scattergl` traces) via `Plotly.js`, which provides the throughput required for clinical datasets sampled at rates of 1,000 Hz or above. Each channel is displayed in a dedicated panel with time expressed in seconds on the horizontal axis ($t = i/f_s$, where i is the sample index and f_s the sampling frequency).

A vertical dashed cursor in every chart is bound to the animation's `frameIndex` and is updated on each render call through `Plotly.layout`, so that as the avatar progresses through the motion, the cursor advances in real time. This coupling allows the clinician to correlate specific postures (such as peak knee flexion, heel strike or toe-off) with corresponding EMG envelope or ground reaction force profile, without manual frame alignment.

This visualization of analog signals is shown in Figure 4.7. The system provides interactive tools for exploring temporal data, including zooming, panning, and value inspection, which are essential for clinical interpretation.

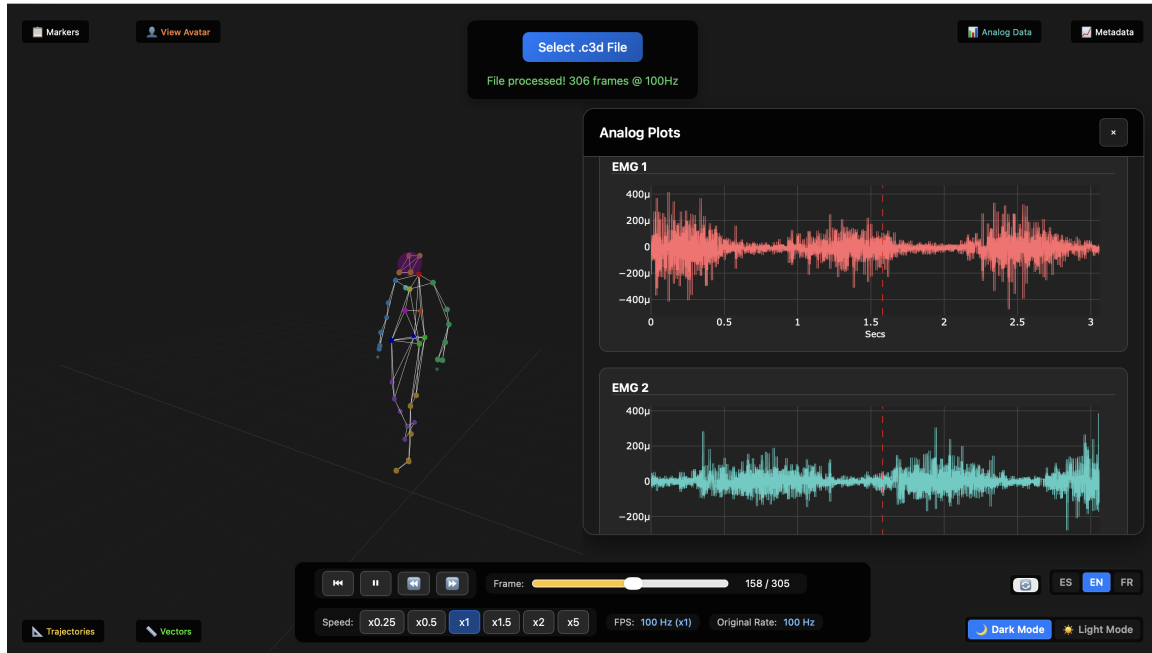


Figure 4.7: Synchronized visualization of 3D motion and EMG signals using a shared temporal reference.

5.1 Validation Methodology

CHLOE validation was structured around two complementary strategies: (a) systematic technical testing using the official C3D test suite provided by `c3d.org` and (b) usability evaluation conducted with clinical professionals. This dual approach ensures both technical correctness of the data ingestion and rendering pipeline and its feasibility for clinical deployment, as specified by the requirements defined in Chapter 3.

The official C3D sample collection [11] available at `c3d.org` is the reference corpus used by the biomechanics community to verify the compatibility of the C3D software. For the purpose of this dissertation, the experimental validation is structured into three dedicated evaluation axes: *Test Suites*, which assess binary format and architecture variants; *Various Manufacturers*, which evaluate multi-vendor interoperability and labeling compatibility; and a series of robustness stress tests specifically designed to evaluate the system’s resilience against parameter formatting errors and malformed data files.

5.2 C3D Format Compatibility Testing (Test Suites)

The test suites evaluate the parser’s ability to handle the full combinatorial space of the C3D binary specification. The format permits data to be stored in three processor endianness

variants: Intel (PC), DEC, and SGI/MIPS, and in two numerical representations (32-bit floating-point and 16-bit integer), yielding six distinct format combinations. A fully compliant implementation must correctly read all six. However, the results obtained with CHLOE revealed a limitation with respect to Big-Endian architectures, which is documented below together with the implemented error handling.

5.2.1 Sample01 - ADTech/DEC and Sample02 - Vicon 370

Sample01 contains six files generated using ADTech AMASS in 1989 on a DEC PDP11 running the RSX11-M operating system, making them the oldest reference implementation of the C3D standard. All six files carry identical biomechanical data: 26 3D points, 16 analog channels, and 450 frames at 50 Hz, encoded in each of the six format variants.

Results: The four Intel and DEC variants (floating-point and integer) were processed successfully. CHLOE returned identical marker counts, frame counts, and analog channel inventories in all four files. However, the two SGI/MIPS variants (Big-Endian format) could not be processed. The `ezc3d` library raised a `RuntimeError` with the message “*MIPS processor type not supported*”.

Rather than allowing this exception to propagate and crash the Flask server, a specific guard was implemented in `processor.py`. The frontend captures the `ARCH_UNSUPPORTED` error type and displays a localized status message, leaving the `Three.js` scene intact. SGI/MIPS files, therefore, result in a controlled and informative failure rather than an application crash. This outcome constitutes a known and documented limitation of CHLOE.

Sample02 reproduces the same six format variants using a Vicon 370 system, with more complex parameter blocks and header events.

Results: Identical to Sample01; Intel and DEC variants loaded correctly, while SGI/MIPS triggered the `ARCH_UNSUPPORTED` guard consistently across both integer and floating-point subtypes, confirming that error isolation is format-agnostic.

5.2.2 Sample08 - Parameter and Data Block Location Variants

Sample08 contains a set of Intel-integer files in which the parameter section and data blocks are placed at deliberately varied byte offsets, exercising the pointer-following logic embedded in the C3D header’s `DATA_START` field.

Results: Success for all files.

CHLOE correctly extracted marker data and parameter metadata regardless of block placement, confirming that the `ezc3d` integration reads the `DATA_START` pointer from the header rather than assuming a fixed offset.

5.2.3 Sample36 - POINT:FRAMES Integer vs. Floating-Point

Sample36 provides three files recording 18,124, 36,220 and 72,610 frames, respectively. In each file, the `POINT:FRAMES` parameter is stored alternately as an integer or a floating-point value, a known C3D standard violation produced by certain data collection software packages.

Results: Files where `POINT:FRAMES` is stored as a valid integer were processed successfully, with all three frame counts correctly extracted and reported in the metadata panel (**FR8**, *Metadata Extraction*). Files where `POINT:FRAMES` is stored as a float caused `ezc3d` to raise a `ValueError` containing the string “*FRAMES is not an INT*”. A specific guard was implemented to intercept this violation.

The frontend displays a localized error message specifying the type violation. The outcome constitutes a partial pass: CHLOE processes compliant large files correctly but explicitly rejects files that violate the parameter type specification, communicating the cause to the user rather than silently producing incorrect frame counts.

5.2.4 Compatibility Test Results

Files with one of these characteristics were not successfully processed:

- SGI/MIPS files. This is a known limitation of the underlying library used and cannot be fixed.
- Files where `POINT:FRAMES` is stored as a float represent behavior that constitutes a violation of the C3D standard. Again, this is outside the scope of the current implementation due to underlying library dependencies.

In both cases, an error message was programmed to gracefully react to the incompatible files. CHLOE properly fulfills **FR1** (*C3D Ingestion*) with a documented exception.

5.3 Current Manufacturers Files (Sample00)

Sample00 is a curated collection submitted by seven manufacturers in 2019 to demonstrate their current C3D implementation. This set directly exercises **FR3** (*Multi-Manufacturer support*), as each file exhibits different marker naming conventions, parameter schemas, coordinate systems, and analog channel configurations. Each file is analyzed individually below because the findings drove concrete implementation decisions.

5.3.1 Vicor Motion Systems

Results: Success for all files.

All markers belonging to the Plug-in-Gait protocol were recognized by `STANDARD_MAP` and correctly mapped to the skeleton. Analysis of the processed files indicated that analog signals are occasionally embedded inside the `POINTS` section of the `C3D` file rather than in the dedicated `ANALOG` section. Vicon’s Plug-in Gait pipeline stores computed time-series quantities (such as joint moments, powers, and angles) using standardized label names, which are gathered in the Plug-in Gait Reference Guide on Vicon’s web [36] [37], within the `POINT` block. CHLOE was extended to detect these labels and reclassify the corresponding data streams as analog channels rather than 3D markers, ensuring that they appear in the analog panel instead of the 3D scene and fulfilling **FR5** (*Analog Signal Plotting*). The first file, `TableTennis.c3d`, served as a primary reference to validate the entire end-to-end pipeline.

5.3.2 NextGen Ergonomics

Results: Markers were recognized and the avatar rendered correctly. However, the subject appears in a lying-down posture in the 3D scene.

This behavior suggests that this software stores data with `Z` as the horizontal propagation axis rather than the vertical. No documentation confirming this convention was found during development. Because all other supported manufacturers use a `Z`-up laboratory frame that is correctly transformed by the `clean_marker_data()` coordinate conversion ($[X, Y, Z] \rightarrow [X, Z, -Y]$), the axis remapping was not altered to avoid breaking existing behavior. A future improvement would be an optional axis-rotation button in the GUI to accommodate non-standard orientations.

5.3.3 Motion Analysis Corporation (MAC)

Results: Success for all files.

Here, it is important to note that a large proportion of the marker labels were virtual markers: computed outputs (e.g., joint centers, segment endpoints) that are derived from raw measurements rather than directly captured from physical markers. These virtual labels are not included in the `STANDARD_MAP` to prevent ambiguity, as some anatomical landmarks exist both as raw physical markers and as computed virtual counterparts. Despite being excluded from the map, virtual markers are not discarded; they are rendered in the scene as gray spheres belonging to the “*Other Markers*” category, remaining visible and interactive for the clinician without interfering with the avatar animation.

5.3.4 Innovative Sports Training (IST) and Codamotion

Results: Success for all files.

Each uses a different proprietary marker nomenclature, which was incorporated into `STANDARD_MAP`. In both cases, analog channels were correctly extracted and displayed.

5.3.5 Advanced Realtime Tracking GmbH (ART)

Results: Processed successfully after identifying that ART exports coordinates in centimeters rather than in millimeters or meters.

The unit-detection heuristic `scalar()` was extended to recognize the ranges of values on the centimeter scale and apply the corresponding $\times 0.01$. ART marker labels were not added to the `STANDARD_MAP` due to the ambiguity of their naming convention, which uses short numeric-suffix identifiers that cannot be reliably mapped to anatomical landmarks. Markers are therefore rendered as the “*Other Markers*” group.

5.3.6 Cometa Systems

Results: Processed successfully after modifying the pipeline.

This file contains analog EMG data but no 3D frame data, a valid use case for wireless EMG systems that do not integrate motion capture. The initial pipeline assumed the presence of at least one frame; the Cometa file exposed this assumption. The processor was updated to handle the analog-only case: If the 3D point section is empty or absent, but analog channels are present, CHLOE continues processing and delivers the analog data to the telemetry dashboard without attempting to populate the 3D scene. This addition broadens CHLOE’s applicability beyond motion capture to pure EMG or force-plate analysis workflows, fulfilling **FR5** (*Analog Signal Plotting*).

5.3.7 Manufacturer Test Results

After modifications, CHLOE successfully processed all manufacturer files, with the exception of the non-standard Z-up files of NextGen Ergonomics, suggesting the support of this feature as a future work. With that said, files with this quirk can still be used, with the caveat of the subject being shown lying down. CHLOE properly fulfills **FR3** (*Multi-Manufacturer support*).

5.4 Robustness and Formatting Issues

5.4.1 Sample12 - Stress Test: 364 Markers, 18,178 Frames

Sample12 is a 100 MB file; processing it revealed an additional non-standard C3D structure: the marker label list was split across two separate parameters, `LABELS` and `LABELS2`,

because the standard LABELS parameter cannot accommodate more than 255 entries in a single block.

Results: Processed successfully after adding concatenation logic.

Without this addition, markers beyond index 255 would have been silently discarded. After the fix, all 364 markers were correctly extracted (fulfilling **FR2**, *3D Visualization*). The Three.js renderer maintained a stable 60 FPS during playback via the delta-time accumulator strategy, and the $O(1)$ frame dictionary lookup remained responsive throughout the session (fulfilling **NFR2**, *Performance*).

5.4.2 Sample13 - Files with Parameter Errors

This collection contains four files.

Results: The first two files triggered the STANDARD_VIOLATION guard (POINT:FRAMES stored as a float), returning a controlled 422 error with a descriptive message, consistent with the behavior of Sample36 (5.2.3). The remaining two files were processed without errors, but the avatar appeared in a lying-down posture because, as documented in the `c3d.org` README, these files use Z as the vertical axis with a different orientation convention.

CHLOE's coordinate transformation ($[X, Y, Z] \rightarrow [X, Z, -Y]$) handles the standard Z-up laboratory frame; a different X-up or non-standard convention is not currently remapped. This is consistent with the observation of Sample00 (NextGen Ergonomics in 5.3.2) and confirms the future need for a user-controlled axis orientation toggle.

5.4.3 Sample16 and Sample20 (Invalid Data)

Files from Sample 16 have all their 3D residuals set to -1, indicating that the data points are invalid, and in Sample 20, there is a 3D data block but no parameters to describe the data.

Results: All files were processed without a crash. CHLOE's `clean_marker_data()` function works correctly. Since no valid frames or analog data were found, the backend returned the following controlled error message in both cases: *Error: The c3d file was processed but contains neither valid frames nor analog data.* The Three.js scene remained stable.

5.4.4 Sample18 - Corrupt Parameter Section

Results: As expected, the corrupt parameter section caused `ezc3d` to raise an exception during parsing. The try-except architecture in `app.py` intercepted the error and returned a 422 response, keeping the WebGL context intact. The frontend displayed the localized error message without any browser freeze.

5.4.5 Sample21 - Data/Parameter Rate Mismatch

The file records data at 200 Hz but `POINT:RATE` claims 60 Hz.

Results: CHLOE extracted and displayed the markers and the analog data without errors. The application uses the frequencies as written in the C3D file, without independently inferring the true sampling rate. As a result, the animation plays at an incorrect speed.

This is a known limitation of trusting the metadata header when no magnitude-based heuristic can unambiguously detect the mismatch. The outcome is documented as an edge case: CHLOE does not crash, but the temporal information displayed in the UI should be treated with caution when a parameter/data mismatch is suspected by the user. However, it can be adjusted using the speed buttons in the playback container.

5.4.6 Robustness Test Results

The experimental evaluation across all five robustness test scenarios demonstrates the overall resilience of the architecture, successfully fulfilling **NFR3** (*Robustness*). Rather than suffering unhandled application crashes or WebGL context losses, CHLOE adapted successfully to non-standard multi-block parameters (5.4.1) and mitigated explicit file corruption or typing violations through its defensive try-except backend architecture (5.4.2, 5.4.3, 5.4.4).

Consequently, the system safely isolates data-driven anomalies, ensuring operational stability and generating clear, localized feedback for the clinical user.

5.5 Clinical and Functional Validation with Physiotherapy Professionals

5.5.1 Usability Tests

To validate CHLOE's usability, the Spanish-language version of the System Usability Scale (SUS) [38] questionnaire was administered to a sample of $N = 7$ physiotherapy professionals. The SUS is a validated 10-item Likert scale that produces a composite score ranging from 0 to 100, widely used as an industry benchmark for usability assessment.

The sample size is considered appropriate for both qualitative and quantitative usability evaluations; according to the mathematical model proposed by Nielsen and Landauer (1993) [39], the involvement of five evaluators allows for the identification of approximately 85% of an interface's usability issues. By including seven participants, the sample not only exceeds this minimum threshold but also enables the collection of satisfaction metrics via the SUS test with a sufficient degree of confidence for expert validation.

The results yielded a mean score of 85.36 ($\sigma = 7.61$), as detailed in Table 5.1. Following

the adjective rating criteria established by Bangor et al. (2008) [40], this places CHLOE in the *Excellent* usability category (Grade A), fulfilling **NFR4** (*Usability*). The low standard deviation indicates high consistency across participants, confirming that the user experience was uniformly positive regardless of the individual’s professional profile or level of technical expertise. All seven participants scored at or above 75 points, substantially exceeding the industry-accepted threshold of 68.

Table 5.1: Individual SUS questionnaire results.

Participant	SUS Score
Expert 1	80.0
Expert 2	85.0
Expert 3	80.0
Expert 4	85.0
Expert 5	75.0
Expert 6	95.0
Expert 7	97.5
Overall Mean	85.36
Standard Deviation (σ)	7.61

Notably, items related to system consistency (P6) and perceived ease of use (P3) received the highest marks, with most experts scoring them at the scale extremes (1 and 5, respectively). This suggests that the application follows recognizable design patterns for healthcare professionals, reducing the cognitive load during navigation.

However, the analysis of P10 (“*I needed to learn a lot of things before I could get going with this system*”) revealed a mean score of 2.57. While still within an acceptable range, the presence of scores such as 3 and 4 indicates a slight learning curve. This finding suggests that although the core functionality is intuitive, some specialized features may require initial guidance. While the visual representation of the moving avatar is designed to be easily interpreted by patients for immediate biofeedback, the analytical layers work as specialized clinical instruments. These features are essential for a physiotherapist’s diagnostic process but may require professional explanation if presented to a patient.

5.5.2 Qualitative Findings

Beyond usability assessment, the clinical sessions also served as functional validation of several system requirements under real deployment conditions.

The application was deployed directly on the clinicians' own computers through the Docker-based distribution workflow, validating **NFR1** (*Portability and Deployment*) by confirming that the system could run consistently across different host environments without manual dependency configuration.

During the evaluation sessions, clinicians dynamically switched between Spanish, English, and French interfaces while preserving the current visualization state, validating **FR7** (*Multi-language Support*). All interface elements, including telemetry panels and controls, updated correctly without requiring page reloads.

NFR5 (*Concurrency and Responsiveness*) was satisfied by architectural design: the production deployment configures Gunicorn with four parallel worker processes, ensuring that concurrent C3D parsing requests from different users do not block each other, without requiring explicit load testing given the reduced number of simultaneous users expected in a clinical deployment context.

Additionally, clinicians tested both skeleton mode and avatar mode during motion playback, validating **FR4** (*Biomechanical Avatar Animation*). The avatar preserved anatomically coherent movement patterns across different gait and upper-limb motion sequences.

Finally, **FR6** (*Temporal Synchronization*) was validated using a cyclic upper-limb movement trial in which the subject repeatedly raised and lowered both arms. During playback, the angular measurements displayed in the vector analysis panel changed coherently with the observed movement. Because the motion sequence was periodic, the plotted angular patterns also repeated cyclically, confirming that the telemetry and the 3D animation shared the same temporal reference throughout the visualization process.

5.5.3 Tested Devices

CHLOE was tested on computers, tablets, and mobile phones. It ran successfully and could be easily used on computers and tablets, supporting compliance with **NFR2** (*Performance*). Concerning mobile phones, although the web application technically worked, the amount of data shown on the screen makes these devices inadequate for this application.

5.6 Summary

The validation process demonstrated that CHLOE satisfies the functional and non-functional requirements defined in Chapter 3 through a combination of systematic technical testing and clinical evaluation.

Testing with the official `c3d.org` corpus confirmed the robustness and interoperability of the ingestion pipeline across different manufacturers, parameter structures, unit systems,

and malformed edge cases. Known limitations, such as the absence of SGI/MIPS compatibility or non-standard axis orientations, were handled through controlled error management without compromising frontend stability.

The validation process also led to several implementation improvements, including specialized guards for unsupported architectures and parameter violations, support for multi-block marker labels, automatic unit scaling, analog-only workflows, and the reclassification of Plug-in Gait computed outputs.

Clinical evaluation with physiotherapy professionals confirmed the practical feasibility of CHLOE in real deployment scenarios. The application achieved a mean SUS score of 85.36, placing it within the *Excellent* usability category. The sessions additionally validated Docker portability, multilingual support, synchronized visualization, and anatomically coherent avatar animation.

Overall, the results indicate that CHLOE is a stable and clinically viable platform for biomechanical data visualization and analysis.

Table 5.2: Traceability matrix: Requirements vs. Validation outcomes.

ID	Requirement	Validation Method	Status
Functional Requirements (FR)			
FR1	C3D Ingestion	Test Suite (Sample01, Sample02, Sample08, Sample36)	PASS*
FR2	3D Visualization	Stress Test (Sample12) & Manufacturer Files	PASS
FR3	Multi-Manufacturer	Manufacturer Suite (Sample00: all 7 vendors)	PASS
FR4	Avatar Animation	Clinical Usability Sessions (Skeleton & Avatar Modes)	PASS
FR5	Analog Signal Plotting	EMG/GRF Samples (Vicon, Cometa Workflow)	PASS
FR6	Synchronization	Cyclic Upper-Limb Trial & Clinical Usability Sessions	PASS
FR7	Multi-language	Professional Evaluation (Dynamic UI Switching)	PASS
FR8	Metadata Extraction	Parsing Verification (Metadata Dashboard Display)	PASS
Non-Functional Requirements (NFR)			
NFR1	Portability (Docker)	Multi-Environment Clinician Deployment	PASS
NFR2	Performance	Rendering Benchmarks (60 FPS with Sample12) & Devices	PASS
NFR3	Robustness	Malformed Samples (Sample13, Sample16, Sample18, Sample20, Sample21)	PASS
NFR4	Usability	SUS Questionnaire Evaluation ($N = 7$, Mean: 85.36)	PASS
NFR5	Concurrency	Production deployment with Unicorn (4 worker processes)	PASS

**Note: FR1 is classified as PASS with the documented exception of SGI/MIPS Big-Endian architectures, as detailed in Section 5.2.*

Conclusions and Future Work

6.1 Conclusions

This Bachelor Thesis describes the design and implementation of CHLOE (Clinical Helper for Locomotion Objective Evaluation), a web-based system for the visualization and analysis of biomechanical data in physiotherapy contexts. The proposed solution addresses the “clinical gap” identified in Chapter 1 by providing an alternative to traditional biomechanical software, enabling direct interaction with C3D data without requiring complex preprocessing pipelines or specialized hardware and/or software.

From a system perspective, CHLOE implements a complete end-to-end pipeline, from C3D ingestion and normalization to real-time visualization and synchronized analytical representation. The backend architecture, based on Python and the `ezc3d` library, parses and standardizes heterogeneous biomechanical datasets. The frontend, developed using Three.js and Plotly.js, enables real-time rendering of marker trajectories, biomechanically consistent avatar animation based on the Plug-in Gait protocol, and interactive visualization of analog signals such as EMG and ground reaction forces. In addition, the system incorporates real-time geometric analysis through vector-based angle computation relative to global reference axes. The use of Docker containerization provides portability and reproducibility across all deployment environments.

Beyond the planned scope, three specific technical characteristics of the data format and

infrastructure altered the initial architecture design:

The first was the discovery that Vicon’s Plug-in Gait pipeline embeds computed time-series quantities (joint moments, powers, and segment angles) inside the POINTS block of the C3D file rather than in the dedicated ANALOG section, using standardized label names. This caused these signals to be erroneously rendered as 3D markers in early versions of the system. Detecting and reclassifying these channels required cross-referencing the Vicon Plug-in Gait documentation and extending the ingestion pipeline with a label-based heuristic, and it ultimately improved the system’s behavior for all Vicon files.

A second challenge arose from the deployment infrastructure. The `ezc3d` library, which wraps a C++ parser, could not be installed via `pip` on ARM-based processors (including Apple Silicon machines and several server architectures) due to the absence of compatible precompiled binaries. The solution, switching to Mamba and the `conda-forge` channel to obtain architecture-specific precompiled packages, resolved the problem but required a two-stage Docker build strategy. This experience reinforced the importance of decoupling the build environment from the runtime environment in any project that depends on compiled native extensions.

A third finding concerned the limits of trusting C3D metadata. Several files in the official test suite contained headers with structural discrepancies: Frame counts stored as floating-point values in violation of the standard, and sampling rates that did not match the number of samples present. Consequently, the processing pipeline was designed with an explicit error-catching architecture to handle type violations and surface informative error messages to the user, which proved essential for handling the full diversity of real-world manufacturer files.

The validation process quantifies the technical behavior and clinical applicability of the system. Technical validation, performed using official C3D test suites and datasets from seven different manufacturers, demonstrates format compatibility across different file structures, robustness against formatting inconsistencies, and data integrity under corrupted conditions. The one documented limitation (the inability to parse SGI/MIPS Big-Endian files) is handled as a controlled, informative failure rather than an unhandled crash, and is classified as a future development item.

In parallel, usability validation with physiotherapy professionals confirmed that the system fulfills its intended clinical purpose. The results indicate that CHLOE provides an intuitive interface, reduces the complexity of biomechanical data interpretation, and facilitates both clinical analysis and educational use cases. Participants indicated that the system has strong potential both as a support tool in daily practice and as a resource in educational settings. In particular, the ability to quickly visualize movement without requiring complex preprocessing was perceived as a significant advantage over existing tools. Notably,

the motor imagery application (using the synchronized avatar to support neurological rehabilitation exercises) emerged spontaneously as a suggestion from clinical participants. This represents an additional application case that had not been originally prioritized, suggesting that the system's utility extends beyond its initial design scope.

6.2 Future Work

Despite the successful fulfillment of the defined requirements, several extensions can be considered for future work.

At the data management level, the integration of a persistent database of C3D files would enable structured storage, efficient querying, and longitudinal tracking of patient data. This would also support the creation of standardized datasets for research purposes and potential machine learning applications.

At the visualization level, the integration of immersive technologies through WebXR represents a developmental path to explore. This would allow the exploration of biomechanical data in extended reality environments, improving spatial perception of movement and enabling advanced interaction paradigms for clinical analysis and rehabilitation scenarios. In this context, particularly for motor imagery applications, an additional line of work would involve generating a subject-specific avatar mesh directly from marker data through inverse kinematics techniques. This approach would enable a higher degree of personalization and realism, potentially improving patient engagement and the effectiveness of visualization-based therapies.

From an analytical perspective, future developments could incorporate reference or normative values within the graphical interface. By leveraging large public biomechanical datasets, it would be possible to compute representative baseline curves (e.g., average gait patterns) to provide clinicians with a direct comparative reference against patient data, supporting objective clinical decision-making.

Finally, future developments may consider the integration of cloud-based services and collaborative tools, enabling remote access, data sharing, and multi-user interaction, but always respecting the relevant Data Protection and Privacy regulations present in the region in which this system would be deployed. This would position CHLOE as a scalable platform for telemedicine and remote biomechanical assessment.

Impact of the Project

A.1 Social Impact

CHLOE has a direct positive social impact in the fields of healthcare and rehabilitation. By democratizing access to biomechanical data recorded in C3D format through a web-based interface that requires no specialized configuration, the system reduces the gap between research laboratories and daily clinical practice. Physiotherapists working in centers that lack the resources to acquire proprietary software (like Visual3D or Vicon Nexus) can now access objective movement analysis without additional investment.

The use of CHLOE in educational contexts was also highlighted by the professionals who participated in the usability tests, who recognized its potential for motor imagery exercises in neurological rehabilitation. This is a use case that was not originally prioritized in the functional requirements but emerged spontaneously during clinical validation, suggesting that the system's utility extends beyond its initial design scope.

The complete source code of the project has been made available for free under a MIT license, ensuring that anyone can freely use and expand it.

A.2 Ethical and Privacy Considerations

CHLOE processes human movement data that may be considered sensitive biometric information. However, the system’s architecture minimizes the associated risks: all data processing is performed locally on the institution’s own infrastructure through Docker containers, without the need to transmit patient data to external servers. This design decision was explicitly motivated by data governance concerns in healthcare contexts, and it represents a clear advantage over cloud-hosted alternatives.

Any clinical deployment of CHLOE must comply with applicable data protection regulations, in particular the General Data Protection Regulation (GDPR) in the European Union, ensuring informed patient consent, appropriate anonymization of C3D files where required, and secure data storage policies. These considerations are especially relevant given the biometric nature of motion capture recordings.

A.3 Economic Impact

In the short term, CHLOE introduces no additional costs for centers that already operate motion capture systems, as it functions as a free, open-source alternative viewer to commercial tools such as Visual3D or Vicon Nexus. In the medium and long term, reducing dependence on proprietary software licenses can represent significant savings for clinical institutions.

Within the broader LibreMotion framework, the integration of low-cost devices such as inertial sensors or XR headsets can dramatically reduce the investment required to access objective biomechanical assessment, potentially opening this type of analysis to a much wider range of physiotherapy centers that would otherwise be excluded due to cost barriers.

A.4 Environmental Impact

The environmental impact of the project is limited. As a web application deployed via lightweight Docker containers on existing infrastructure, CHLOE requires no specialized hardware or additional equipment beyond what institutions already operate. The system’s browser-based nature means it can run on any modern device, avoiding the need for dedicated workstations.

The primary environmental cost is the energy consumption of the servers on which the application is deployed, which is comparable to that of any small-scale web application. On the positive side, the possibility of performing biomechanical assessments remotely, reducing travel for both patients and specialists, may contribute modestly to a reduction in transport-

related carbon emissions.

A.5 Professional Responsibility

Given that CHLOE is intended for use in clinical environments, professional responsibility in its design and validation is fundamental. The system has been conceived as a support tool for clinical interpretation, not as a diagnostic device. This distinction is important: CHLOE facilitates objective visualization of movement, but the final clinical interpretation always remains the responsibility of the healthcare professional.

Technical validation using the official `c3d.org` test suite and usability testing with physiotherapy professionals provide a reasonable level of correctness and fitness for clinical use. However, any future extension of the system towards automated quantitative analysis (like automated detection of pathological gait patterns) would require more rigorous clinical validation before being used to support medical decision-making. Healthcare professionals deploying CHLOE must ensure that its limitations are clearly communicated to both colleagues and patients.

Economic Budget

B.1 Technological Resources

The following technological resources were used during the development of the project:

- **Development computer** (laptop with ARM processor): Approximately 1,200 €.
- **Deployment server** (local instance or cloud environment used for testing): Marginal cost, covered by existing academic infrastructure.

All software tools used throughout the project (Python, Flask, Three.js, Plotly.js, Docker, Node.js, Vite, and `ezc3d`) are open-source and freely available. As a result, no software licensing costs were incurred.

B.2 Human Resources

The Bachelor Thesis carries an academic workload of 12 ECTS credits, equivalent to approximately 360 hours of work. Taking the average gross salary of a junior Biomedical Engineer in Spain, which ranges between 22,000 € and 28,000 € per year [41], results in an average profile of approximately 25,000 € per year or roughly 14 €/hour, the estimated cost of human resources is:

$$360 \text{ hours} \times 14 \text{ €/hour} = \mathbf{5,040 \text{ €}}$$

B.3 Total Cost

The total estimated cost of the project is obtained by combining the technological and human resources involved in its development. Table B.1 summarizes the main cost components.

Cost item	Estimated cost (€)
Technological resources	1,200
Human resources	5,040
Total	6,240

Table B.1: Estimated total cost of the project.

It should be noted that the technological resources used in this project correspond to general-purpose computing equipment that may also be used for other academic or professional activities. The estimated cost therefore reflects the use of these resources during the development period rather than an exclusive investment dedicated solely to this project.

Bibliography

- [1] S. L. Delp, F. C. Anderson, et al. Opensim: Open-source software to create and analyze dynamic simulations of movement. *IEEE Transactions on Biomedical Engineering*, 54(11):1940–1950, 2007.
- [2] OpenSim Documentation Team. Opensim documentation - c3d (.c3d) files, 2026.
- [3] OpenSim Documentation Team. Opensim documentation - marker (.trc) files, 2026.
- [4] OpenSim Documentation Team. Opensim documentation - motion (.mot) files, 2026.
- [5] C-Motion Inc. Visual3D: Biomechanical modeling and analysis software, 2026. Accessed: April 2026.
- [6] Vicon Motion Systems Ltd. *Vicon Nexus User Guide*, 2024. Accessed: April 2026.
- [7] Arnaud Barre and Stéphane Armand. Biomechanical ToolKit: Open-source framework to visualize and process biomechanical data. *Computer Methods and Programs in Biomedicine*, 114(1):80–87, 2014.
- [8] C3D.org. The c3d format home page, 2026.
- [9] The Khronos Group. OpenXR Overview.
- [10] CMU Graphics Lab. Cmu graphics lab motion capture database, 2026. Accedido el 16 de enero de 2026.
- [11] C3D.org. The c3d format sample data, 2026.
- [12] Vicon Motion Systems Ltd. Full body modeling with plug-in gait, 2024. Vicon Nexus Documentation.
- [13] Python Software Foundation. c3d: A Python library for reading and writing C3D files. <https://pypi.org/project/c3d/>, 2026. Accessed: May 2026.
- [14] Pyomeca. ezc3d: C3d library, 2026.
- [15] Python Software Foundation. Python programming language, 2026. Official webpage of Python programming language.
- [16] NumPy. Numpy python library, 2026.
- [17] Pallets Projects. Flask documentation, 2026.
- [18] NGINX Authors. Nginx documentation, 2026.
- [19] FastAPI Authors. Fastapi documentation, 2026.

- [20] Django Software Foundation. Django documentation, 2026.
- [21] Gunicorn Developers. Gunicorn: Python wsgi http server for unix, 2026.
- [22] Pallets Projects. Flask documentation - deploying with uwsgi, 2026.
- [23] Three.js Authors. Three.js documentation, 2026.
- [24] Babylon.js Authors. Babylon.js documentation, 2026.
- [25] A-Frame Authors. A-frame documentation, 2026.
- [26] Node.js Authors. Node.js documentation, 2026.
- [27] Vite Contributors. Vite guide: Getting started, 2026.
- [28] Adobe Systems Incorporated. Mixamo: 3d character animation and rigging tool, 2026.
- [29] Chart.js Authors. Chart.js documentation, 2026.
- [30] D3.js Authors. D3.js data-driven documents, 2026.
- [31] Plotly. Plotly javascript graphing library, 2026.
- [32] stack.gl Authors. stack.gl - an open software ecosystem for webgl, 2026.
- [33] Docker Inc. Docker: Accelerated container application development, 2026. Containers' platform for apps deployment.
- [34] Mamba Contributors. Mamba documentation, 2026.
- [35] Anaconda Inc. Anaconda software packages and conda package manager, 2026. Repository of packages for scientific computing and dependency management.
- [36] Vicon Motion Systems Ltd. Plug-in gait lower body forces and moments, 2024. Vicon Nexus Documentation.
- [37] Vicon Motion Systems Ltd. Plug-in gait output angles, 2024. Vicon Nexus Documentation.
- [38] MDR Sevilla-Gonzalez, L Moreno Loaeza, LS Lazaro-Carrera, B Bourguet Ramirez, A Vázquez Rodríguez, ML Peralta-Pedrero, and P Almeda-Valdes. Spanish version of the system usability scale for the assessment of electronic tools: Development and validation, 2020.
- [39] Jakob Nielsen and Thomas K. Landauer. A mathematical model of the finding of usability problems. In *Proceedings of the INTERACT'93 and CHI'93 Conference on Human Factors in Computing Systems*, pages 206–213. ACM, 1993.
- [40] Aaron Bangor, Philip T Kortum, and James T Miller. An empirical evaluation of the system usability scale. *International Journal of Human-Computer Interaction*, 24(6):574–594, 2008.
- [41] Universidad Alfonso X el Sabio. ¿cuál es el salario de un ingeniero biomédico en españa?, 2025.